

figures/EMSI.png



Projet de Fin d'Année

Cybersécurité et Infrastructures Réseau

Conception et réalisation d'une
application mobile, d'une API et d'un
back-office pour un centre de beauté

Réalisé par :

Prénom NOM

Encadré par :

Pr. Prénom NOM, Encadrant Académique, EMSI Marrakech.

Mme/M. Prénom NOM, Encadrant Professionnel, Yani Concept by Fati, Marrakech.

Année Universitaire : 2025–2026

Dédicace

Je dédie ce modeste travail :

À mes parents, pour leurs sacrifices, leur patience et la confiance qu'ils n'ont jamais cessé de m'accorder.

À ma famille et à mes amis, pour leur soutien constant et leurs encouragements tout au long de ces années.

À tous les professeurs de École Marocaine des Sciences de l'Ingénieur, qui ont donné sans compter de leur temps et de leur savoir pour nous former.

À Madame Fati, gérante du centre Yani Concept by Fati, pour m'avoir ouvert les portes de son institut, pour le temps qu'elle a consacré à répondre à mes questions et pour la confiance qu'elle a placée dans ce projet.

À toutes celles et ceux qui, de près ou de loin, ont participé à l'élaboration de ce travail.

Et à toute personne qui prendra ce rapport entre ses mains.

Remerciements

C'est avec une grande satisfaction que je tiens à remercier toutes les personnes qui m'ont aidé, accompagné et soutenu jusqu'à l'aboutissement de ce projet de fin d'année.

J'adresse en premier lieu mes remerciements à l'ensemble du corps professoral de École Marocaine des Sciences de l'Ingénieur, dont les enseignements constituent le socle sur lequel ce travail a pu être bâti. Les notions de génie logiciel, de bases de données, de sécurité et de conduite de projet mobilisées dans les pages qui suivent y ont toutes été acquises.

Je remercie tout particulièrement *Pr. Prénom NOM*, mon encadrant pédagogique, pour son suivi régulier, ses relectures exigeantes et les conseils qui ont orienté ce projet aux moments où il en avait le plus besoin.

Je remercie également *Mme/M. Prénom NOM*, ainsi que Madame Fati, gérante du centre *Yani Concept by Fati*, pour sa disponibilité et pour la qualité des échanges que nous avons eus. C'est en observant le fonctionnement réel de son institut — le carnet de rendez-vous, les appels téléphoniques, la carte de fidélité tamponnée — que les besoins de cette application ont pu être formulés autrement que de façon théorique. Les allers-retours réguliers avec elle ont directement façonné les arbitrages de conception exposés dans ce rapport : c'est à sa demande, par exemple, que la durée d'une prestation a cessé de commander la grille des créneaux.

Mes remerciements vont enfin aux membres du jury, qui me font l'honneur d'accepter d'évaluer ce travail, ainsi qu'à mes parents, dont le soutien indéfectible ne s'est jamais démenti.

Merci à toutes et à tous.

Résumé

Ce rapport de projet de fin d'année présente la conception et la réalisation d'une solution logicielle complète pour *Yani Concept by Fati*, un centre de beauté situé à Marrakech. Le projet répond à une situation courante dans les petites structures de services : une activité entièrement pilotée à la main — carnet de rendez-vous papier, prises de rendez-vous par téléphone et par messagerie instantanée, vente de produits sans suivi de stock, fidélité gérée par une carte tamponnée — avec les conséquences qui en découlent : double réservation d'un même créneau, absence d'historique exploitable, temps consacré au téléphone au détriment des clientes présentes.

La solution développée est un mono-dépôt réunissant trois applications adossées à une même base de données PostgreSQL. Une **API REST** bâtie avec NestJS et Prisma porte l'ensemble des règles métier. Une **application mobile** développée avec React Native et Expo permet aux clientes de consulter les catalogues, de réserver une prestation, de commander des produits et de suivre leur compte de fidélité. Un **back-office** web en React donne à la gérante et à son personnel la maîtrise des commandes, des rendez-vous, du catalogue, des horaires d'ouverture et du programme de fidélité.

Le rapport détaille l'ensemble du cycle de développement : étude de l'existant, expression des besoins, modélisation UML, choix techniques, architecture en couches, implémentation et validation. Une méthode agile inspirée de SCRUM, organisée en six sprints, a permis de livrer par incréments et de confronter chaque version au fonctionnement réel de l'institut.

Un chapitre entier est consacré à la **sécurité** de la plate-forme. Partant d'une délimitation de la surface d'attaque et d'un modèle de menaces STRIDE, il expose les contre-mesures effectivement implémentées : authentification par Argon2id et jetons courts, rotation des jetons de rafraîchissement avec révocation de famille en cas de réutilisation détectée, défense contre l'énumération de comptes par uniformisation du temps de réponse, contrôle d'accès par rôles doublé d'une vérification de propriété des ressources, validation stricte des entrées, reconnaissance des fichiers téléversés par leur

signature binaire, limitation du débit et plafonnement des volumes, et anonymisation transactionnelle des comptes supprimés — réponse au droit à l’effacement de la loi 09-08 sans sacrifier l’historique comptable de l’institut. Les limites subsistantes y sont inventoriées.

Une attention particulière a par ailleurs été portée aux propriétés que les tests unitaires classiques ne savent pas démontrer : sérialisation des réservations concurrentes par verrou consultatif PostgreSQL, atomicité du crédit de fidélité, idempotence de l’émission des bons de récompense. Ces propriétés relèvent autant de la correction que de la sécurité, une condition de course déclenchable à volonté étant une vulnérabilité. Vingt-cinq suites de tests, dont huit écrivent dans une véritable base PostgreSQL, sont rejouées à chaque modification par une chaîne d’intégration continue.

Le résultat est une plate-forme fonctionnelle, trilingue (français, arabe, anglais), documentée et prête au déploiement, qui supprime le carnet papier, rend la réservation possible à toute heure et donne à l’institut une visibilité qu’il n’avait pas sur son activité.

Mots clés : sécurité applicative, modèle de menaces, STRIDE, Argon2id, JWT, rotation de jetons, contrôle d’accès par rôles, loi 09-08, anonymisation, application mobile, API REST, NestJS, Prisma, PostgreSQL, React Native, Expo, React, TypeScript, réservation en ligne, programme de fidélité, concurrence, intégration continue.

Abstract

This end-of-year project report presents the design and implementation of a complete software solution for *Yani Concept by Fati*, a beauty centre located in Marrakech. The project addresses a situation common to small service businesses: an activity managed entirely by hand — a paper appointment book, bookings taken over the phone and through instant messaging, product sales with no stock tracking, and a loyalty programme run on a stamped card — with the consequences that follow: the same slot booked twice, no usable history, and time spent on the phone at the expense of the customers already in the salon.

The delivered solution is a monorepo bringing together three applications backed by a single PostgreSQL database. A **REST API** built with NestJS and Prisma carries all the business rules. A **mobile application** built with React Native and Expo lets customers browse the catalogues, book a treatment, order products and follow their loyalty account. A **web back-office** written in React gives the manager and her staff control over orders, appointments, the catalogue, opening hours and the loyalty programme.

The report covers the full development cycle: study of the existing process, requirements elicitation, UML modelling, technical choices, layered architecture, implementation and validation. An agile method inspired by SCRUM, organised into six sprints, made it possible to deliver in increments and to confront each version with the real workings of the salon.

A full chapter is devoted to the **security** of the platform. Starting from an attack-surface analysis and a STRIDE threat model, it sets out the countermeasures actually implemented: Argon2id password hashing and short-lived access tokens, refresh-token rotation with family-wide revocation on detected reuse, defence against account enumeration by equalising response times, role-based access control backed by per-resource ownership checks, strict input validation, magic-byte recognition of uploaded files, rate limiting and volume caps, and transactional anonymisation of deleted accounts — the

answer to the right to erasure under Moroccan law 09-08 without sacrificing the salon's accounting history. Remaining limitations are inventoried.

Particular care was also given to properties that ordinary unit tests cannot demonstrate: serialisation of concurrent bookings through a PostgreSQL advisory lock, atomicity of loyalty crediting, and idempotent issuance of reward vouchers. These belong as much to security as to correctness, since a race condition that can be triggered at will is a vulnerability. Twenty-five test suites, eight of which write to a real PostgreSQL database, are replayed on every change by a continuous integration pipeline.

The result is a working, trilingual (French, Arabic, English), documented platform, ready for deployment, which removes the paper book, makes booking possible at any hour, and gives the salon a visibility over its own activity that it previously lacked.

Keywords: application security, threat modelling, STRIDE, Argon2id, JWT, token rotation, role-based access control, law 09-08, anonymisation, mobile application, REST API, NestJS, Prisma, PostgreSQL, React Native, Expo, React, TypeScript, online booking, loyalty programme, concurrency, continuous integration.

Glossaire

Sigle	Signification
API	<i>Application Programming Interface</i> — interface de programmation applicative
Argon2	Fonction de dérivation de clé, lauréate du <i>Password Hashing Competition</i>
CI	<i>Continuous Integration</i> — intégration continue
CNDP	Commission Nationale de contrôle de la protection des Données à caractère Personnel
COD	<i>Cash On Delivery</i> — paiement à la remise ou à la livraison
CORS	<i>Cross-Origin Resource Sharing</i> — partage de ressources entre origines
CSP	<i>Content Security Policy</i> — politique de sécurité du contenu
CSPRNG	<i>Cryptographically Secure Pseudo-Random Number Generator</i>
CSRF	<i>Cross-Site Request Forgery</i> — falsification de requête intersite
CVE	<i>Common Vulnerabilities and Exposures</i> — référencement public des vulnérabilités
DTO	<i>Data Transfer Object</i> — objet de transfert de données
EAS	<i>Expo Application Services</i> — chaîne de compilation native d'Expo
HSTS	<i>HTTP Strict Transport Security</i>
HTTP(S)	<i>HyperText Transfer Protocol (Secure)</i>
i18n	<i>Internationalization</i> — internationalisation
IANA	<i>Internet Assigned Numbers Authority</i> — source de la base des fuseaux horaires
IDOR	<i>Insecure Direct Object Reference</i> — accès direct non contrôlé à une ressource
IHM	Interface Homme-Machine
JSON	<i>JavaScript Object Notation</i>
JWT	<i>JSON Web Token</i> — jeton d'authentification signé
MFA	<i>Multi-Factor Authentication</i> — authentification à plusieurs facteurs
ORM	<i>Object-Relational Mapping</i> — correspondance objet-relationnel
OWASP	<i>Open Worldwide Application Security Project</i>
PFA	Projet de Fin d'Année
RBAC	<i>Role-Based Access Control</i> — contrôle d'accès fondé sur les rôles
REST	<i>REpresentational State Transfer</i> — style d'architecture d'API
RFC	<i>Request For Comments</i> — document normatif de l'IETF

Liste des figures

1.1	Identité visuelle de <i>Yani Concept by Fati</i>	3
1.2	Organisation du centre et acteurs du système	5
1.3	Le cycle de la méthode agile SCRUM	11
1.4	Le cycle du développement piloté par les tests	13
1.5	Diagramme de Gantt du projet	15
2.1	Diagramme de cas d'utilisation général de la plate-forme	21
2.2	Diagramme de cas d'utilisation — gestion des rendez-vous	21
2.3	Diagramme de cas d'utilisation — commande de produits	21
2.4	Diagramme de cas d'utilisation — programme de fidélité	22
2.5	Diagramme de cas d'utilisation — back-office	22
2.6	Diagramme de séquence — inscription et vérification de l'adresse	27
2.7	Diagramme de séquence — réservation d'un créneau	27
2.8	Diagramme de séquence — commande et décrétement du stock	28
2.9	Diagramme de séquence — clôture d'un rendez-vous, crédit de points et palier	29
2.10	Diagramme de séquence — rafraîchissement et rotation des jetons	29
2.11	Diagramme de classes de la plate-forme	30
3.1	Les couches de l'application	39
3.2	Architecture technique et déploiement de la solution	39
4.1	Surface d'attaque et frontières de confiance	43
4.2	Chaîne de traitement de sécurité d'une requête	49
5.1	Structure des modules du backend	64
5.2	Structure du back-office	65
5.3	Structure de l'application mobile	66
5.4	Exécution de la chaîne d'intégration continue	69
5.5	Résultat de l'exécution complète de la suite de tests	69
5.6	Écran de connexion · Écran d'inscription	69

5.7	Vérification de l'adresse · Mot de passe oublié	70
5.8	Écran d'accueil · Catalogue des prestations	70
5.9	Détail d'une prestation · Catalogue des produits	70
5.10	Choix de la date et du créneau · Récapitulatif avant validation	71
5.11	Confirmation du rendez-vous · Mes rendez-vous	71
5.12	Détail d'un produit · Panier	71
5.13	Validation de commande · Mes commandes	72
5.14	Écran de fidélité · Mes récompenses et bons	72
5.15	Profil et réglages · Modification du profil	72
5.16	Back-office — écran de connexion	73
5.17	Back-office — tableau de bord	73
5.18	Back-office — gestion des commandes	73
5.19	Back-office — gestion des rendez-vous	74
5.20	Back-office — gestion du catalogue	74
5.21	Back-office — programme de fidélité	74
5.22	Back-office — horaires et fermetures	75
5.23	Back-office — gestion des utilisateurs	75
5.24	Back-office — fenêtre d'export Excel	75
5.25	Courriel de vérification d'adresse	76

Liste des tableaux

1.1	Les acteurs de l’institut et leurs responsabilités	5
1.2	Réponse apportée à chaque difficulté identifiée	9
1.3	Répartition des rôles SCRUM sur le projet	12
1.4	Découpage du projet en sprints	14
2.1	Besoins non fonctionnels et moyens de satisfaction	19
2.2	Description du cas d’utilisation « Réserver une prestation »	23
2.3	Description du cas d’utilisation « Passer une commande »	24
2.4	Description du cas d’utilisation « Échanger des points contre une récompense »	25
2.5	Description du cas d’utilisation « Traiter une commande »	26
3.1	Répartition des routes de l’API par domaine	36
3.2	Environnement matériel de développement	37
3.3	Environnement logiciel de développement	38
4.1	Modèle de menaces STRIDE appliqué à la plate-forme	44
4.2	Protection des codes à usage unique	47
4.3	Matrice des droits par rôle	49
4.4	Principaux en-têtes de sécurité posés	53
4.5	Limites de débit par catégorie de route	54
4.6	Synthèse menaces → contre-mesures	60
5.1	Organisation du mono-dépôt	63
5.2	Répartition des suites de tests par domaine	67
5.3	Travaux de la chaîne d’intégration continue	68

Table des matières

Dédicace	i
Remerciements	ii
Résumé	iii
Abstract	v
Glossaire	viii
Liste des figures	x
Liste des tableaux	xii
Table des matières	xiii
Introduction Générale	1
1 Présentation de l'organisme d'accueil et du contexte du projet	3
Introduction	3
1.1 Présentation de l'organisme d'accueil	3
1.1.1 Le centre <i>Yani Concept by Fati</i>	3
1.1.2 Activités et offre de services	4
1.1.3 Organisation et acteurs	4
1.2 Présentation du contexte du projet	5
1.2.1 Vue d'ensemble du projet	5
1.2.2 Étude de l'existant	6
1.2.3 Problématique	7
1.2.4 Solution proposée	7
1.2.5 Enjeux du projet	9
1.2.6 Contraintes	10
1.3 Conduite du projet	11

1.3.1	La méthode agile SCRUM	11
1.3.2	Répartition des rôles	11
1.3.3	Réunions	12
1.3.4	Le développement piloté par les tests	12
1.3.5	Les sprints	13
1.3.6	Planification du projet	14
	Conclusion	15
2	Analyse et conception	16
	Introduction	16
2.1	Spécifications des besoins	16
2.1.1	Besoins fonctionnels	16
2.1.2	Besoins non fonctionnels	18
2.1.3	Acteurs du système	19
2.1.4	Langage de modélisation	20
2.2	Diagrammes de cas d'utilisation	20
2.2.1	Diagramme de cas d'utilisation général	20
2.2.2	Cas d'utilisation : réservation d'une prestation	21
2.2.3	Cas d'utilisation : commande de produits	21
2.2.4	Cas d'utilisation : programme de fidélité	21
2.2.5	Cas d'utilisation : administration	22
2.3	Description des cas d'utilisation	22
2.4	Diagrammes de séquence	26
2.4.1	Inscription et vérification de l'adresse e-mail	27
2.4.2	Réservation d'un créneau	27
2.4.3	Traitement d'une commande et gestion du stock	28
2.4.4	Clôture d'un rendez-vous et crédit de fidélité	29
2.4.5	Rotation des jetons de session	29
2.5	Diagramme de classes	30
2.5.1	Vue d'ensemble	30
2.5.2	Choix structurants du modèle	30
	Conclusion	31
3	Étude technique	33
	Introduction	33
3.1	Technologies de développement	33
3.1.1	Langages	33
3.1.2	Cadriciels et bibliothèques	34
3.1.3	Services web	34
3.1.4	Système de gestion de base de données	36

3.1.5	Outils de développement	37
3.2	Architecture technique	38
3.2.1	Architecture en couches	38
3.2.2	Architecture de déploiement	39
3.2.3	Place de la sécurité dans l'architecture	40
	Conclusion	41
4	Sécurité de la plate-forme	42
	Introduction	42
4.1	Démarche et modèle de menaces	42
4.1.1	Surface d'attaque	42
4.1.2	Modèle de menaces	43
4.1.3	Principes directeurs	44
4.2	Authentification	45
4.2.1	Stockage des mots de passe	45
4.2.2	Jetons d'accès	45
4.2.3	Jetons de rafraîchissement, rotation et détection de vol	46
4.2.4	Codes à usage unique	47
4.2.5	Défense contre l'énumération de comptes	47
4.3	Autorisation et contrôle d'accès	48
4.3.1	Modèle de contrôle d'accès	48
4.3.2	Application par gardes successifs	49
4.3.3	Contrôle de propriété des ressources	50
4.3.4	Invariants d'administration	50
4.4	Sécurité des entrées et des sorties	50
4.4.1	Validation stricte des entrées	51
4.4.2	Injection SQL	51
4.4.3	Contrôle des fichiers téléversés	51
4.4.4	Fuite d'information par les messages d'erreur	52
4.4.5	Injection d'en-tête de courriel	52
4.5	Sécurité du transport et des en-têtes	53
4.5.1	Chiffrement du transport	53
4.5.2	En-têtes de sécurité HTTP	53
4.5.3	Contrôle des origines	54
4.6	Disponibilité et résistance à l'abus	54
4.6.1	Limitation du débit	54
4.6.2	Plafonnement des volumes	55
4.6.3	Correction sous concurrence	55
4.7	Traçabilité et non-répudiation	56

4.8	Protection des données personnelles	56
4.8.1	Cadre applicable	56
4.8.2	L'effacement par anonymisation	57
4.8.3	Gestion des secrets	58
4.8.4	Sauvegardes	58
4.9	Sécurité de la chaîne de développement	59
4.10	Synthèse	59
4.11	Limites connues et durcissement à venir	61
	Conclusion	61
5	Mise en œuvre	63
	Introduction	63
5.1	Structure du projet	63
5.1.1	Le backend	64
5.1.2	Le back-office	65
5.1.3	L'application mobile	65
5.2	Tests et validation	66
5.2.1	Stratégie de tests	66
5.2.2	Le test de concurrence, en détail	67
5.2.3	Intégration continue	68
5.2.4	Résultats	69
5.3	Interfaces graphiques	69
5.3.1	Application mobile	69
5.3.2	Back-office	73
5.3.3	Courriers électroniques	76
5.4	Déploiement et exploitation	76
5.4.1	Mise en production	76
5.4.2	Sauvegardes	76
5.4.3	Ce qui reste à faire avant une ouverture publique	76
	Conclusion	77
	Conclusion Générale et Perspectives	78
	Webographie	82

Introduction Générale

La transformation numérique n'est plus l'apanage des grandes entreprises. Elle touche aujourd'hui les commerces de proximité, les cabinets, les artisans et les instituts, dont l'activité repose encore très souvent sur des outils manuels : un carnet, un téléphone, une carte cartonnée. Ces outils ont une qualité que l'on sous-estime — ils sont immédiats, personne n'a besoin de les apprendre — et un défaut qui devient handicapant dès que l'activité croît : ils ne conservent rien d'exploitable, ne se consultent qu'à un seul endroit, et n'existent que pendant les heures d'ouverture.

Le secteur de la beauté et du bien-être illustre ce constat de façon particulièrement nette. L'activité d'un institut s'organise autour d'une ressource rare et non stockable — un créneau de cabine — que plusieurs canaux se disputent en même temps : la cliente qui appelle, celle qui écrit sur une messagerie instantanée, et celle qui se présente au comptoir. Le carnet papier arbitre entre ces canaux, mais il ne peut le faire que si quelqu'un le tient ouvert devant soi. Toute réservation prise ailleurs qu'à côté du carnet est une réservation à risque.

C'est dans ce contexte que s'inscrit ce projet de fin d'année. *Yani Concept by Fati* est un centre de beauté installé à Marrakech, qui propose des prestations de soin et vend des produits cosmétiques. Sa gérante y assure à la fois les soins, la prise des rendez-vous, le suivi du stock et la fidélisation de sa clientèle. L'objectif du projet est de porter l'ensemble de ces activités sur une plate-forme numérique : une application mobile pour les clientes, un back-office web pour l'institut, et une API qui porte les règles métier communes aux deux.

Le travail présenté ici ne s'est pas limité à traduire un carnet papier en formulaires. Une réservation en ligne pose des questions qu'un carnet ne pose pas : que se passe-t-il lorsque deux clientes valident le même créneau à la même seconde ? Comment un institut ouvert de 9 h à 13 h puis de 14 h à 18 h, à l'heure de Marrakech, expose-t-il ses disponibilités à une application qui raisonne en temps universel ? Que devient l'historique d'achat d'une cliente qui exerce son droit à l'effacement, alors même que cet historique porte le chiffre d'affaires de l'institut ? Ces questions ont structuré la

conception autant que les fonctionnalités elles-mêmes, et elles occupent une place proportionnée dans ce rapport.

Ces questions expliquent aussi la place qu'occupe la sécurité dans ce rapport. Une application accessible depuis les téléphones de clientes, sur des réseaux quelconques, n'a pas seulement des utilisatrices : elle a des visiteurs. Elle détient par ailleurs des données personnelles et porte des opérations à valeur économique — des points de fidélité convertibles en prestations gratuites. La sécurité n'a donc pas été traitée comme une couche ajoutée en fin de projet, mais comme une dimension de chaque décision de conception ; elle fait l'objet d'un chapitre entier.

Le présent document rend compte de ce travail. Il s'articule autour de cinq chapitres.

Le **premier chapitre** présente l'organisme d'accueil, l'étude de l'existant, la problématique qui en découle et la solution retenue, puis la méthode de conduite de projet et sa planification.

Le **deuxième chapitre** est consacré à l'analyse et à la conception : la spécification des besoins fonctionnels et non fonctionnels, les acteurs du système, puis la modélisation UML — diagrammes de cas d'utilisation, diagrammes de séquence et diagramme de classes.

Le **troisième chapitre** expose l'étude technique : les langages, les cadrage et le système de gestion de base de données retenus, les environnements de travail, et l'architecture technique de la solution, y compris la chaîne de sécurité appliquée à chaque requête.

Le **quatrième chapitre** est consacré à la sécurité de la plate-forme : la surface d'attaque et le modèle de menaces retenu, puis les contre-mesures mises en œuvre — authentification, contrôle d'accès, validation des entrées et des fichiers, sécurité du transport, résistance à l'abus, traçabilité et protection des données personnelles au regard de la loi 09-08. Il se termine par une synthèse et par l'inventaire des limites qui subsistent.

Le **cinquième chapitre** traite de la mise en œuvre : la structure effective du code des trois applications, la stratégie de tests et l'intégration continue, la présentation des interfaces réalisées, et enfin le déploiement et l'exploitation de la plate-forme.

Une conclusion générale dresse le bilan du travail accompli, ses limites assumées et les perspectives d'évolution de la solution.

CHAPITRE 1

Présentation de l'organisme d'accueil et du contexte du projet

Introduction

Ce chapitre présente le cadre dans lequel ce projet de fin d'année a été réalisé. Il décrit d'abord l'organisme d'accueil — son activité, son organisation et les personnes qui y travaillent — puis le contexte du projet proprement dit : la façon dont l'institut fonctionnait avant l'intervention, les difficultés que ce fonctionnement engendrait, la solution retenue pour y répondre, ainsi que les enjeux et les contraintes qui l'ont encadrée. Il se termine par la méthode de conduite de projet adoptée et par la planification qui en découle.

1.1 Présentation de l'organisme d'accueil

1.1.1 Le centre *Yani Concept by Fati*

Yani Concept by Fati est un centre de beauté et de bien-être installé à Marrakech. C'est une structure de proximité : une clientèle essentiellement féminine, largement fidélisée, qui revient à intervalles réguliers et qui connaît personnellement la gérante. Cette proximité est le principal atout commercial de l'institut ; c'est aussi ce qui rend la question de l'outil informatique délicate, car toute solution qui interposerait une machine froide entre la cliente et l'institut détruirait précisément ce qui fait sa valeur.



Figure 1.1 : Identité visuelle de *Yani Concept by Fati*

L'identité visuelle du centre — un or lumineux sur fond crème ou noir profond — a été reprise à l'identique dans les trois applications développées. Ce n'est pas un détail cosmétique : pour une cliente, l'application doit manifestement être celle de l'institut qu'elle connaît, et non un outil générique de réservation dans lequel l'institut ne serait qu'une entrée parmi d'autres.

1.1.2 Activités et offre de services

L'activité du centre se répartit en deux volets, qui ont chacun leur logique propre et qui sont tous deux couverts par la solution développée.

- **Les prestations de soin.** Elles constituent le cœur de métier. Chaque prestation appartient à une catégorie (soins du visage, soins du corps, épilation, coiffure, ongles...), porte un tarif et s'effectue en cabine, sur rendez-vous. La ressource limitante est le nombre de cabines réservables simultanément.
- **La vente de produits cosmétiques.** L'institut revend des produits de soin. Cette activité obéit à une logique différente : elle porte sur un stock physique, se règle à la remise ou à la livraison, et n'occupe aucune cabine.

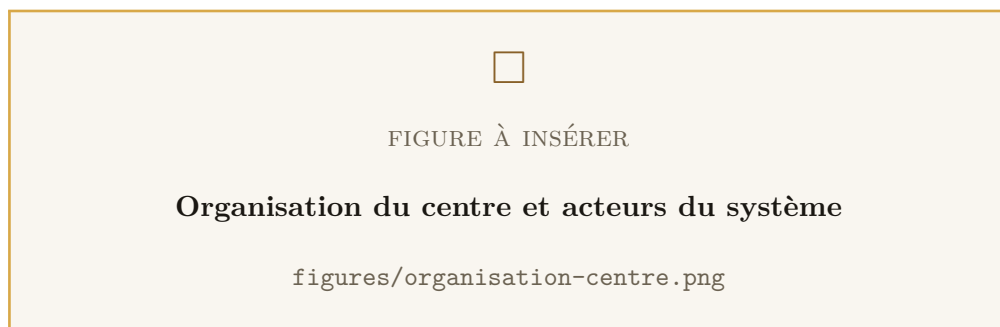
À ces deux volets s'ajoute un **programme de fidélité**, transversal, qui récompense à la fois les prestations réalisées et les produits achetés.

1.1.3 Organisation et acteurs

L'institut est une structure de petite taille, ce qui a une conséquence directe sur la conception : les rôles ne sont pas cloisonnés comme ils le seraient dans une entreprise plus grande, et une même personne assure souvent plusieurs fonctions dans la même journée. Trois profils ont néanmoins été distingués, parce qu'ils correspondent à trois niveaux de responsabilité réellement différents.

Tableau 1.1 : Les acteurs de l'institut et leurs responsabilités

Acteur	Rôle système	Responsabilités
La gérante	ADMIN	Direction du centre. Fixe les tarifs et le catalogue, règle les horaires d'ouverture et la capacité, définit le programme de fidélité, gère les comptes du personnel et consulte les exports comptables.
Le personnel	STAFF	Réalise les soins et tient le comptoir. Traite les commandes, confirme et clôture les rendez-vous, honore les bons de récompense, crédite exceptionnellement des points à la main.
Les clientes	CLIENT	Consultent les catalogues, réservent leurs rendez-vous, commandent des produits et suivent leur compte de fidélité.

**Figure 1.2** : Organisation du centre et acteurs du système

Cette hiérarchie de rôles n'est pas une simple convention documentaire : elle est appliquée par le serveur sur chaque route de l'API, indépendamment de ce que l'interface propose ou masque. Une cliente qui obtiendrait par un moyen quelconque l'adresse d'une route d'administration se verrait refuser l'accès par le serveur lui-même. Ce point est développé au chapitre 4.

1.2 Présentation du contexte du projet

1.2.1 Vue d'ensemble du projet

Le projet consiste à concevoir et à réaliser une solution logicielle complète couvrant l'ensemble de l'activité de l'institut. Elle prend la forme d'un mono-dépôt réunissant trois applications qui partagent une seule base de données :

- une **API REST** qui porte les données et la totalité des règles métier ;
- une **application mobile** destinée aux clientes ;
- un **back-office web** destiné à la gérante et à son personnel.

Le choix de faire porter toutes les règles métier par l'API, et par elle seule, est structurant. Les deux interfaces — mobile et web — n'implémentent aucune règle : elles affichent ce que le serveur leur donne et lui soumettent ce que l'utilisatrice saisit. Une règle telle que « on ne peut pas réserver dans le passé » ou « un rendez-vous annulé ne peut plus être marqué terminé » n'existe qu'à un seul endroit. Ce principe évite la divergence classique où l'application mobile et le back-office finissent par appliquer des règles subtilement différentes à la même donnée.

1.2.2 Étude de l'existant

Avant tout développement, une observation du fonctionnement réel de l'institut a été menée. Elle a mis en évidence quatre processus manuels, tous fonctionnels au quotidien, et tous porteurs des mêmes limites.

La prise de rendez-vous. Elle s'effectue par trois canaux concurrents : le téléphone, la messagerie instantanée et la présentation directe au comptoir. La réservation est inscrite au crayon dans un carnet papier, unique et physiquement situé à l'accueil. La cliente ne dispose d'aucune confirmation écrite autre que la mémoire de l'échange.

La vente de produits. Elle se fait exclusivement sur place. Le stock n'est suivi par aucun outil : la gérante l'estime de visu. Un produit épuisé ne se découvre qu'au moment où une cliente le demande.

La fidélité. Elle repose sur une carte cartonnée tamponnée à chaque visite. La carte perdue emporte l'historique avec elle ; la carte oubliée à la maison prive la cliente de son tampon ; et l'institut n'a aucune vision consolidée de ce que le programme lui coûte ni de ce qu'il lui rapporte.

Le suivi de l'activité. Il n'existe pas de manière systématique. Il n'est pas possible de répondre, sans reconstitution manuelle, à des questions aussi élémentaires que « combien de rendez-vous ce mois-ci ? », « quelles sont les prestations les plus demandées ? » ou « quelles clientes ne sont pas revenues depuis six mois ? ».

1.2.3 Problématique

Ce fonctionnement, viable tant que l'activité reste modeste, engendre des difficultés qui se renforcent mutuellement à mesure qu'elle croît.

- **Le risque de double réservation.** Trois canaux alimentent un carnet unique. Une réservation prise par téléphone pendant qu'une cliente écrit sur la messagerie peut viser le même créneau. Le conflit ne se découvre qu'au moment où les deux clientes se présentent, et sa résolution se fait toujours au détriment de l'une d'elles.
- **L'indisponibilité hors des heures d'ouverture.** Une cliente qui souhaite réserver le soir ou le dimanche doit attendre. Une partie de ces demandes ne se transforme jamais en rendez-vous.
- **Le temps consacré au téléphone.** Chaque appel interrompt un soin en cours. Ce coût est invisible dans les comptes, mais il est réel : il se paie en qualité de service pour la cliente présente en cabine.
- **L'absence de mémoire exploitable.** Le carnet et la carte de fidélité ne conservent qu'une trace éphémère et non consolidée. L'institut ne peut donc ni mesurer son activité, ni identifier ses prestations phares, ni détecter l'érosion de sa clientèle.
- **La fragilité du support physique.** Un carnet se perd, s'abîme, et n'existe qu'en un seul exemplaire, à un seul endroit.
- **La rupture de stock invisible.** Sans suivi, un produit épuisé n'est découvert qu'au comptoir, devant la cliente.

Ces difficultés ont un dénominateur commun : *l'information de l'institut n'existe qu'en un seul exemplaire, dans un seul lieu, et pendant les seules heures d'ouverture.* C'est cette contrainte que le projet lève.

1.2.4 Solution proposée

La solution retenue est une plate-forme composée de trois applications adossées à une base de données unique. Ce choix mérite d'être justifié : il aurait été possible de se contenter d'un back-office web, ou de recourir à une plate-forme de réservation existante. Ni l'une ni l'autre de ces options n'a été retenue.

Un back-office seul aurait déplacé le carnet papier vers un écran, sans rien changer pour la cliente : elle aurait continué d'appeler. Une plate-forme de réservation généraliste, quant à elle, aurait réglé la question des créneaux mais placé l'institut au milieu d'un

annuaire de concurrents, sans couvrir ni la vente de produits ni un programme de fidélité propre — et en faisant dépendre l'activité d'un tiers sur lequel l'institut n'a aucune prise.

La plate-forme développée comprend donc :

- **Une API REST (NestJS, Prisma, PostgreSQL).** Elle expose plus de quatre-vingts routes et porte l'ensemble des règles métier : catalogue, moteur de créneaux, machine à états des rendez-vous et des commandes, programme de fidélité, authentification, exports.
- **Une application mobile (React Native, Expo).** Vingt-quatre écrans permettent à la cliente de parcourir les catalogues, de réserver, de commander, de suivre ses points et de gérer son compte. Elle est disponible en français, en arabe et en anglais.
- **Un back-office web (React, Vite).** Huit pages donnent à l'institut la maîtrise complète de son activité : tableau de bord, commandes, rendez-vous, catalogue, fidélité, horaires, utilisateurs, avec export Excel des tableaux réservé à la gérante.

Tableau 1.2 : Réponse apportée à chaque difficulté identifiée

Difficulté	Réponse apportée
Double réservation d'un créneau	Canal de réservation unique, et sérialisation des réservations concurrentes par verrou consultatif PostgreSQL : deux demandes simultanées sur le même créneau sont traitées l'une après l'autre, jamais en parallèle.
Indisponibilité hors ouverture	Réservation possible à toute heure depuis l'application, sur la grille de créneaux calculée par le serveur à partir des horaires réels du centre.
Temps passé au téléphone	La réservation en autonomie décharge l'accueil ; le rendez-vous pris par téléphone reste possible, le personnel le saisissant pour la cliente.
Absence de mémoire	Historique complet en base, tableau de bord, et exports Excel horodatés des clientes, des commandes et des rendez-vous.
Fragilité du support papier	Base PostgreSQL, script de sauvegarde horodatée avec rotation à quatorze jours et procédure de restauration documentée.
Rupture de stock invisible	Stock suivi par produit, contrôlé à la commande, décrémenté de façon atomique à la confirmation, et alerte de stock faible sur le tableau de bord.
Carte de fidélité perdue	Compte de fidélité attaché au compte de la cliente : solde, historique, paliers de visites et bons de récompense, consultables depuis l'application.

1.2.5 Enjeux du projet

- **Enjeu commercial.** Rendre la réservation possible en dehors des heures d'ouverture, et transformer en rendez-vous des demandes aujourd'hui perdues faute d'interlocuteur disponible.
- **Enjeu de fidélisation.** Remplacer une carte cartonnée fragile par un compte consultable, doté de paliers de visites et de bons de récompense traçables — un programme que l'institut peut enfin piloter et chiffrer.
- **Enjeu de pilotage.** Donner à la gérante une visibilité qu'elle n'a jamais eue

sur son activité : commandes en attente, rendez-vous du jour, stocks faibles, historique exportable.

- **Enjeu d'image.** Une application aux couleurs de l'institut, soignée et trilingue, participe directement à la perception de la marque auprès d'une clientèle jeune et connectée.
- **Enjeu de conformité.** Traiter des données personnelles impose des obligations, notamment le droit à l'effacement prévu par la loi 09-08 et contrôlé par la CNDP. Ce droit devait être réellement exerçable, et non simplement annoncé.

1.2.6 Contraintes

Contrainte d'exactitude sur les créneaux. Un créneau de cabine ne peut pas être vendu deux fois. Cette contrainte, banale à énoncer, est la plus difficile du projet : la vérification de disponibilité et l'insertion du rendez-vous portent sur un *ensemble* de lignes, ce qu'aucune écriture conditionnelle sur une ligne unique ne sait rendre atomique.

Contrainte de fuseau horaire. L'institut vit à l'heure de Marrakech, dont le décalage varie dans l'année. Les instants sont stockés en UTC, les horaires d'ouverture exprimés en heure locale. Toute confusion entre les deux se traduit par une grille de créneaux décalée d'une heure — une erreur silencieuse, qui ne lève aucune exception et qui ne se voit qu'en comparant l'écran au carnet.

À noter, car cela surprend à la lecture du code : le fuseau est déclaré sous le nom **Africa/Casablanca**. Ce n'est pas une erreur ni un reliquat. La base IANA nomme chaque zone d'après sa ville la plus peuplée, et cette zone couvre l'ensemble du Maroc, Marrakech comprise. La renommer serait la casser.

Contrainte de conservation de l'historique. Une cliente peut demander la suppression de son compte. Ses commandes et ses rendez-vous portent pourtant le chiffre d'affaires de l'institut, qui doit lui survivre. Concilier les deux exige une anonymisation, et non une suppression en cascade.

Contrainte d'usage. L'utilisatrice principale du back-office n'est pas informatienne. L'outil devait être immédiatement compréhensible, sans formation ni documentation à consulter — ce qui a proscrit toute exposition de valeurs techniques brutes dans l'interface comme dans les exports.

Contrainte de coût. L'institut est une petite structure. L'ensemble de la solution repose donc sur des technologies libres et sur une infrastructure minimale, déployable

sur un seul serveur.

Contrainte de délai. Le projet devait aboutir à une plate-forme fonctionnelle et déployable dans la durée impartie au projet de fin d'année, soit environ huit semaines de développement effectif.

1.3 Conduite du projet

1.3.1 La méthode agile SCRUM

Le choix d'une méthode de développement conditionne l'organisation de tout le projet. La méthode agile SCRUM a été retenue, pour une raison qui tient au contexte plus qu'à la mode : les besoins de l'institut n'étaient pas formalisables à l'avance. Ils se sont précisés à mesure que des versions concrètes étaient montrées à la gérante.

L'exemple le plus net concerne le moteur de créneaux. La première version calculait les disponibilités à partir de la durée de chaque prestation : une prestation d'une heure vingt-cinq occupait le temps correspondant. Le comportement était techniquement juste, et fonctionnellement inutilisable — il produisait des heures de rendez-vous irrégulières, impossibles à annoncer au téléphone. La gérante espace en réalité ses rendez-vous d'un écart fixe, quelle que soit la prestation. Le moteur a donc été refait : un rendez-vous occupe exactement un créneau, et la durée de la prestation n'est plus qu'une information de gestion. Aucune spécification écrite à l'avance n'aurait produit cette règle ; seule la confrontation d'une version fonctionnelle au métier réel l'a fait apparaître.

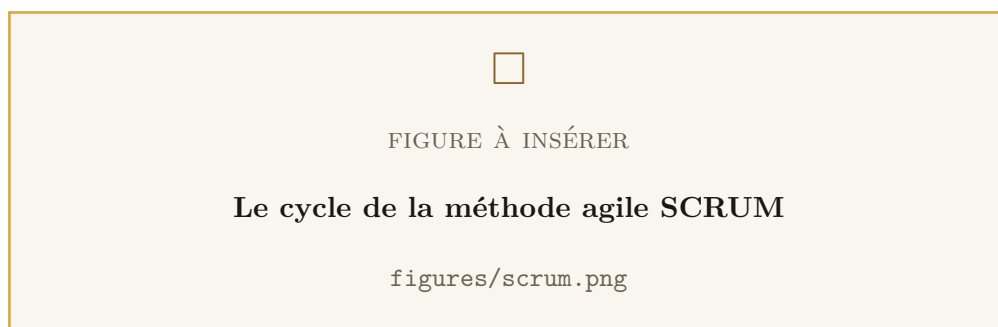


Figure 1.3 : Le cycle de la méthode agile SCRUM

1.3.2 Répartition des rôles

Le projet ayant été mené par un seul développeur, la répartition canonique des rôles SCRUM a été adaptée sans être abandonnée — car ce sont bien trois responsabilités distinctes, même lorsque deux d'entre elles sont portées par la même personne.

Tableau 1.3 : Répartition des rôles SCRUM sur le projet

Rôle	Tenu par	Responsabilité
<i>Product Owner</i>	Madame Fati, gérante	Exprime le besoin, arbitre les priorités, valide chaque incrément à l'aune du fonctionnement réel de l'institut.
<i>SCRUM Master</i>	<i>Pr. Prénom NOM</i>	Cadence les revues, lève les obstacles et veille au respect de la méthode.
<i>Équipe de développement</i>	<i>Prénom NOM</i>	Conception, développement des trois applications, tests, documentation et déploiement.

1.3.3 Réunions

Trois rendez-vous ont rythmé chaque sprint.

- **La planification de sprint.** En début de sprint, les éléments les plus prioritaires du *product backlog* sont sélectionnés pour constituer le *sprint backlog*.
- **La revue de sprint.** En fin de sprint, l'incrément est démontré : l'application est installée sur le téléphone de la gérante et le back-office ouvert devant elle. C'est là que les écarts entre le besoin exprimé et le besoin réel se révèlent.
- **La rétrospective.** Immédiatement après la revue, elle porte sur la façon de travailler et non sur le produit. C'est de l'une d'elles qu'est née la règle appliquée dans tout le projet : *tout comportement non évident doit être expliqué dans le code, à l'endroit où il s'applique*. Cette règle explique la densité inhabituelle de commentaires du dépôt, et elle a été décisive lors des reprises après interruption.

1.3.4 Le développement piloté par les tests

Une approche inspirée du TDD a été appliquée, non pas uniformément, mais là où elle apporte le plus : les règles métier délicates, et tout particulièrement les comportements en situation de concurrence.

Écrire d'abord le test qui prouve que trois réservations simultanées ne peuvent pas occuper deux cabines oblige à formuler la propriété avant de choisir la technique. C'est ce test, écrit contre une véritable base PostgreSQL, qui a imposé le recours à un verrou consultatif : la solution intuitive — compter puis insérer — le faisait échouer de façon reproductible.

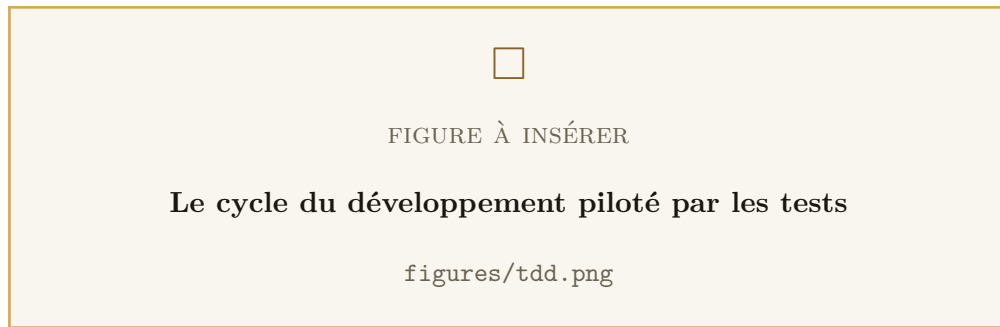


Figure 1.4 : Le cycle du développement piloté par les tests

Le même cycle a été appliqué au crédit de fidélité, à l'émission des bons de récompense, à la rotation des jetons de session et à l'anonymisation des comptes. Ces cinq domaines concentrent l'essentiel des huit suites de tests qui écrivent réellement en base.

1.3.5 Les sprints

Le projet s'est déroulé sur six sprints, du 6 juillet au 27 août 2026.

Tableau 1.4 : Découpage du projet en sprints

Sprint	Période	Contenu livré
1	6 – 9 juillet	Socle de l'API : authentification JWT, catalogues de prestations et de produits, module de rendez-vous, horaires d'ouverture, fidélité, récompenses, gestion des utilisateurs, jetons de rafraîchissement, gestion du fuseau du centre, documentation Swagger.
2	12 – 14 juillet	Application mobile cliente : catalogues, réservation, fidélité, profil, rendez-vous. Extraction des composants réutilisables et refonte graphique complète aux couleurs de l'institut.
3	16 – 23 juillet	Commandes de produits avec paiement à la remise, et back-office complet : tableau de bord, commandes, rendez-vous, catalogue, fidélité, utilisateurs.
4	7 – 11 août	Durcissement : vérification de l'adresse e-mail, réinitialisation du mot de passe, récompenses de palier, écritures atomiques sur le solde, le stock et les créneaux, anonymisation des comptes supprimés, identité native de l'application mobile.
5	12 – 18 août	Réglages du centre et fermetures exceptionnelles, contrôle du stock, pagination des listes, finitions d'expérience utilisateur, exports Excel réservés à la gérante, internationalisation en français, arabe et anglais.
6	21 – 27 août	Hébergement local des images du catalogue, optimisation du décodage des images, mise à jour du script de sauvegarde, conteneurisation et finalisation de la documentation d'exploitation.

1.3.6 Planification du projet

La planification a été établie avant le démarrage puis ajustée à chaque revue. Le diagramme de Gantt ci-dessous en donne la vue d'ensemble : les phases d'analyse et de conception y précèdent le développement, mais elles ne s'y substituent pas — conformément à la démarche agile, une part de conception a été refaite à chaque sprint, au vu de ce que la revue précédente avait révélé.



Figure 1.5 : Diagramme de Gantt du projet

Conclusion

Ce chapitre a permis de cerner le contexte du projet : un institut de beauté dont l'activité, entièrement pilotée à la main, souffrait d'une information unique, localisée et périssable. La solution retenue — une API portant les règles métier, une application mobile pour les clientes et un back-office pour l'institut — a été justifiée, ses enjeux et ses contraintes énoncés, et la méthode de conduite du projet exposée. Le chapitre suivant traduit ces besoins en une analyse et une conception détaillées.

CHAPITRE 2

Analyse et conception

Introduction

Ce chapitre présente le résultat du travail d’analyse et de conception. Il commence par la spécification des besoins — fonctionnels puis non fonctionnels — et par l’identification des acteurs du système. Il expose ensuite la modélisation UML retenue : les diagrammes de cas d’utilisation, leur description textuelle, les diagrammes de séquence des processus les plus délicats, et enfin le diagramme de classes qui fixe la structure des données.

Une précision liminaire sur la méthode : les modèles présentés ici ne sont pas des artefacts produits en amont puis abandonnés. Ils ont été révisés à chaque sprint, et le diagramme de classes en particulier reflète l’état final d’un schéma qui a connu vingt migrations successives. Les choix qui l’ont fait évoluer sont explicités en fin de chapitre, car ils constituent l’essentiel de l’apport de conception de ce projet.

2.1 Spécifications des besoins

2.1.1 Besoins fonctionnels

Les besoins fonctionnels décrivent ce que le système doit permettre de faire. Ils ont été regroupés en huit domaines.

Gestion des comptes et de l’authentification.

- Inscription d’une cliente avec adresse e-mail, mot de passe, nom, prénom et téléphone.
- Confirmation de l’adresse e-mail par un code à usage unique.
- Connexion, déconnexion et maintien de session sans reconnexion quotidienne.
- Réinitialisation du mot de passe oublié, par code envoyé par e-mail.
- Consultation et modification du profil, changement de mot de passe.
- Suppression du compte à la demande de la cliente.

Catalogue de prestations.

- Consultation publique des prestations actives, par catégorie.

- Consultation du détail d'une prestation : description, tarif, photo.
- Création, modification, activation et désactivation des prestations et de leurs catégories par la gérante.

Catalogue de produits.

- Consultation publique des produits actifs, par catégorie.
- Affichage de la disponibilité en stock.
- Gestion complète du catalogue et des stocks par la gérante.

Gestion des rendez-vous.

- Consultation des créneaux disponibles pour une prestation à une date donnée.
- Réservation d'un créneau par la cliente.
- Consultation de ses propres rendez-vous, à venir et passés.
- Annulation par la cliente.
- Réservation *pour le compte* d'une cliente par le personnel, pour les rendez-vous pris par téléphone ou au comptoir.
- Confirmation, clôture, annulation et report d'un rendez-vous par le personnel.

Gestion des commandes.

- Panier et passage de commande, avec retrait à l'institut ou livraison à domicile.
- Paiement à la remise ou à la livraison.
- Consultation de ses commandes et de leur avancement par la cliente.
- Annulation par la cliente tant que la commande n'a pas été remise.
- Traitement des commandes par le personnel selon un cycle de vie contrôlé.

Programme de fidélité.

- Gain automatique de points sur les prestations réalisées et les commandes remises.
- Consultation du solde et de l'historique des mouvements.
- Catalogue de récompenses échangeables contre des points.
- Paliers de visites offrant automatiquement une récompense, avec option de ré-currence.
- Émission d'un bon de récompense doté d'un code lisible, honoré au comptoir.
- Ajout manuel de points par le personnel, avec motif obligatoire et journal consultable par la gérante.

Paramétrage du centre.

- Définition des horaires d'ouverture, à raison de plusieurs plages par jour.
- Déclaration de fermetures exceptionnelles (congés, jours fériés).
- Réglage du nombre de places réservables simultanément et de l'écart entre créneaux, sans redéploiement.

Pilotage et exports.

- Tableau de bord : commandes en attente, rendez-vous du jour, alertes de stock faible.
- Gestion des comptes du personnel et attribution des rôles.
- Export au format Excel des clientes, des commandes et des rendez-vous, réservé à la gérante.

2.1.2 Besoins non fonctionnels

Les besoins non fonctionnels portent sur les qualités attendues de la solution plutôt que sur ses fonctions. Sept ont été retenus, chacun assorti du moyen concret par lequel il est satisfait — un besoin non fonctionnel qui ne se traduit par aucune décision d'implémentation n'est qu'une intention.

Tableau 2.1 : Besoins non fonctionnels et moyens de satisfaction

Besoin	Moyen retenu
Exactitude	Aucun créneau ni aucune unité de stock ne peut être attribué deux fois. Les réservations concurrentes sont sérialisées par un verrou consultatif PostgreSQL ; le stock et le solde de points sont modifiés par écritures atomiques conditionnelles.
Sécurité	Mots de passe hachés avec Argon2 ; jetons d'accès JWT de courte durée ; jetons de rafraîchissement stockés hachés, avec rotation et détection de réutilisation ; autorisation par rôle vérifiée côté serveur sur chaque route ; en-têtes de sécurité HTTP ; validation stricte de toute entrée ; limitation du débit par adresse IP.
Fiabilité	Aucune erreur technique n'atteint l'utilisatrice : un filtre d'exceptions global traduit les erreurs de base de données en messages compréhensibles. Les états des rendez-vous et des commandes sont régis par des machines à états explicites qui interdisent toute transition incohérente.
Performance	Toutes les listes susceptibles de croître sont paginées côté serveur ; le tableau de bord ne demande que ce qu'il affiche ; les images du catalogue sont servies avec un cache d'un an et décodées à la taille réellement affichée.
Ergonomie	Charte graphique unifiée entre les trois applications, modes clair et sombre, messages en langue naturelle, aucun terme technique exposé.
Accessibilité linguistique	Interface mobile disponible en français, en arabe et en anglais ; les réponses de l'API suivent l'en-tête Accept-Language de la requête et les e-mails la langue choisie par la cliente.
Conformité	Droit à l'effacement réellement exercable, par anonymisation du compte plutôt que par suppression en cascade, afin de préserver l'historique comptable de l'institut (loi 09-08, CNDP).

2.1.3 Acteurs du système

Cinq acteurs interagissent avec le système : quatre acteurs humains — la visiteuse non authentifiée et les trois rôles applicatifs — et un acteur secondaire non humain.

- **La visiteuse** — acteur non authentifié. Elle consulte les catalogues publics et peut créer un compte. Elle ne peut ni réserver ni commander.

- **La cliente (CLIENT)** — acteur principal de l'application mobile. Elle réserve, commande, suit son compte de fidélité et gère son profil. Elle n'a aucun accès au back-office : une tentative de connexion y est refusée à l'entrée.
- **Le personnel (STAFF)** — acteur principal du back-office. Il traite les commandes et les rendez-vous, réserve pour le compte d'une cliente, honore les bons de récompense et crédite manuellement des points.
- **La gérante (ADMIN)** — elle dispose de tous les droits du personnel, auxquels s'ajoutent la maîtrise du catalogue, des horaires, des réglages du centre, du programme de fidélité, des comptes utilisateurs, du journal des ajouts manuels de points et des exports Excel.
- **Le service de messagerie** — acteur secondaire non humain. Il achemine les codes de vérification d'adresse et de réinitialisation de mot de passe.

2.1.4 Langage de modélisation

Le langage UML (*Unified Modeling Language*) a été retenu pour modéliser le système. C'est un langage graphique normalisé, indépendant de tout processus de développement et de tout langage de programmation, ce qui permet de l'intégrer sans friction à une démarche agile : les diagrammes sont révisés à chaque sprint plutôt que figés au départ.

Trois types de diagrammes ont été employés, chacun pour ce qu'il montre le mieux :

- les **diagrammes de cas d'utilisation**, pour délimiter le périmètre fonctionnel et répartir les droits entre acteurs ;
- les **diagrammes de séquence**, pour décrire l'ordre des échanges dans les processus où cet ordre est précisément ce qui fait la correction du système ;
- le **diagramme de classes**, pour fixer la structure des données persistantes et leurs relations.

2.2 Diagrammes de cas d'utilisation

2.2.1 Diagramme de cas d'utilisation général

La figure ci-dessous présente la vue d'ensemble des cas d'utilisation du système, tous acteurs confondus.

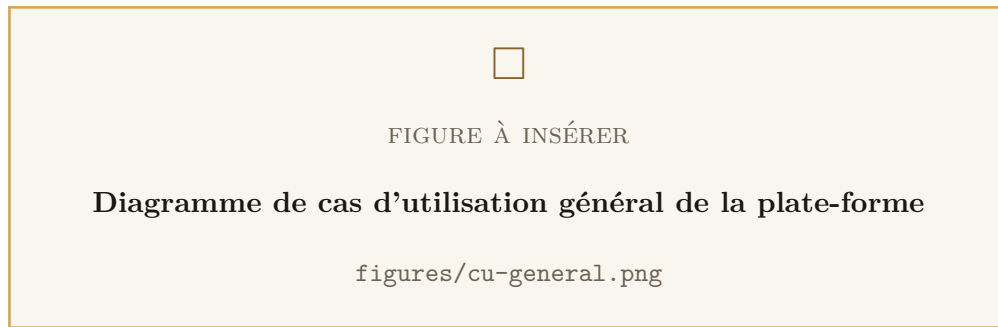


Figure 2.1 : Diagramme de cas d'utilisation général de la plate-forme

2.2.2 Cas d'utilisation : réservation d'une prestation

Le domaine de la réservation est le plus riche du système. Il fait intervenir la cliente et le personnel sur des cas partiellement communs : le personnel peut réserver pour le compte d'une cliente, ce qui est le même cas d'utilisation exercé par un autre acteur, mais il dispose en outre du report et du changement de statut.

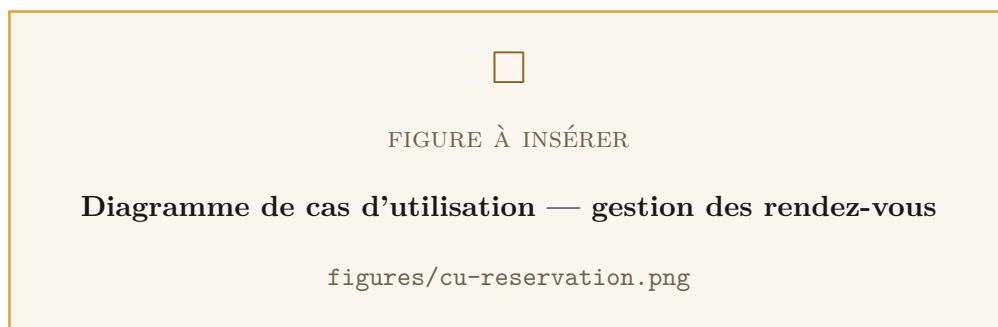


Figure 2.2 : Diagramme de cas d'utilisation — gestion des rendez-vous

2.2.3 Cas d'utilisation : commande de produits

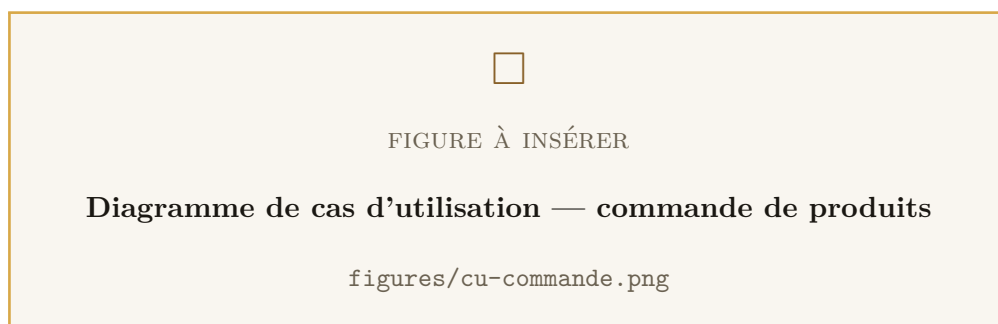


Figure 2.3 : Diagramme de cas d'utilisation — commande de produits

2.2.4 Cas d'utilisation : programme de fidélité

Le programme de fidélité comporte deux circuits de récompense bien distincts, qui convergent vers un même objet : le bon de récompense. Le premier est automatique :

un palier de visites franchi débloquent une récompense que la cliente réclame. Le second est volontaire : la cliente échange ses points contre une récompense du catalogue. Dans les deux cas, un bon est émis, et c'est ce bon que le personnel honore au comptoir.

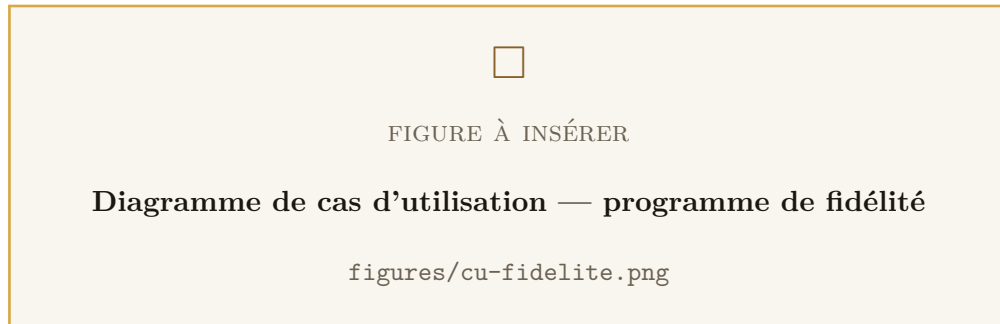


Figure 2.4 : Diagramme de cas d'utilisation — programme de fidélité

2.2.5 Cas d'utilisation : administration

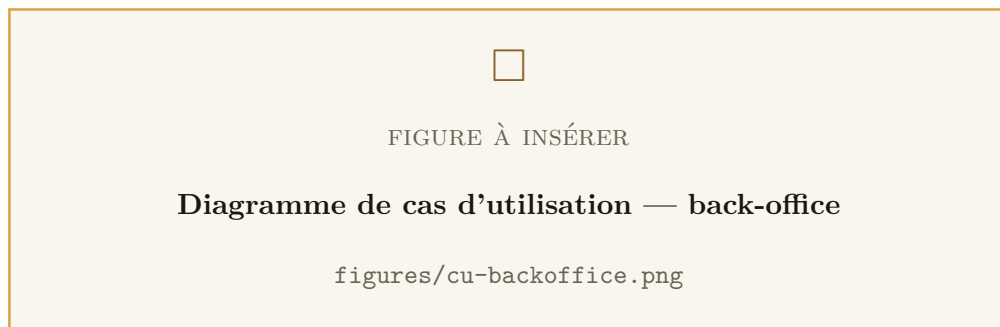


Figure 2.5 : Diagramme de cas d'utilisation — back-office

2.3 Description des cas d'utilisation

Les cas d'utilisation les plus structurants sont décrits ci-après sous forme textuelle. Les scénarios alternatifs y occupent une place volontairement importante : ce sont eux qui déterminent la robustesse du système, et plusieurs d'entre eux n'ont été identifiés qu'à l'usage.

Tableau 2.2 : Description du cas d'utilisation « Réserver une prestation »

Cas d'utilisation	Réserver une prestation
Acteur principal	La cliente
Objectif	Obtenir un rendez-vous pour une prestation, à une date et une heure disponibles
Préconditions	La cliente est authentifiée ; la prestation est active au catalogue
Scénario nominal	1. La cliente sélectionne une prestation et une date. 2. Le système calcule la grille des créneaux du jour à partir des plages d'ouverture, de l'écart réglé par le centre et des fermetures exceptionnelles. 3. Le système marque comme indisponible tout créneau dont la capacité est atteinte, ainsi que tout créneau déjà passé. 4. La cliente choisit un créneau libre et valide. 5. Le système prend le verrou de réservation, contrôle à nouveau la disponibilité, enregistre le rendez-vous au statut <i>en attente</i> et fige le tarif de la prestation. 6. Le système affiche la confirmation.
Alternatives	2a. Le jour est fermé (aucune plage, ou fermeture exceptionnelle) : le système l'indique et ne propose aucun créneau. 4a. Le créneau a été pris entre l'affichage et la validation : le système refuse et invite à en choisir un autre. 5a. La date demandée est passée : le système refuse la réservation.
Postconditions	Le rendez-vous existe au statut <i>en attente</i> ; la capacité du créneau est diminuée d'une unité

Tableau 2.3 : Description du cas d'utilisation « Passer une commande »

Cas d'utilisation	Passer une commande de produits
Acteur principal	La cliente
Objectif	Commander des produits, avec retrait à l'institut ou livraison à domicile
Préconditions	La cliente est authentifiée ; son panier n'est pas vide
Scénario nominal	1. La cliente ajoute des produits à son panier et ouvre la validation. 2. Elle choisit le mode de remise ; en cas de livraison, elle saisit une adresse. 3. Le système fusionne les lignes en double, contrôle que chaque produit est actif et que le stock est suffisant. 4. Le système fige le prix unitaire de chaque ligne, calcule le total et enregistre la commande au statut <i>en attente</i>. 5. La cliente reçoit le récapitulatif de sa commande.
Alternatives	2a. Livraison choisie sans adresse : le système refuse. 3a. Un produit a été retiré du catalogue ou son stock est insuffisant : le système refuse la commande en désignant le produit en cause.
Postconditions	La commande existe au statut <i>en attente</i> ; <i>le stock n'est pas encore décrémenté</i> — il ne l'est qu'à la confirmation par le personnel

Ce dernier point mérite d'être souligné, car il résulte d'un arbitrage explicite. Décrémenter le stock dès la commande aurait permis de le réserver, mais aurait ouvert la porte au blocage du stock par des commandes jamais honorées. Le paiement s'effectuant à la remise, l'institut confirme chaque commande par un appel : c'est ce moment, et non la commande elle-même, qui engage réellement le produit.

Tableau 2.4 : Description du cas d'utilisation « Échanger des points contre une récompense »

Cas d'utilisation	Échanger des points contre une récompense
Acteur principal	La cliente
Objectif	Convertir un solde de points en récompense utilisable à l'institut
Préconditions	La cliente est authentifiée ; la récompense est active au catalogue
Scénario nominal	1. La cliente consulte le catalogue des récompenses et son solde. 2. Elle demande l'échange d'une récompense. 3. Le système débite le solde de façon atomique, conditionnellement à sa suffisance. 4. Le système enregistre le mouvement et émet un bon de récompense muni d'un code lisible. 5. La cliente présente ce code au comptoir lors de sa prochaine visite.
Alternatives	3a. Le solde est devenu insuffisant entre l'affichage et la demande : le débit conditionnel n'affecte aucune ligne, l'échange est refusé et rien n'est émis.
Postconditions	Le solde est diminué du coût de la récompense ; un bon dû figure sur la liste du comptoir

Tableau 2.5 : Description du cas d'utilisation « Traiter une commande »

Cas d'utilisation	Traiter une commande
Acteur principal	Le personnel
Objectif	Faire progresser une commande jusqu'à sa remise à la cliente
Préconditions	Le membre du personnel est authentifié au back-office avec le rôle STAFF ou ADMIN
Scénario nominal	1. Le personnel ouvre la liste des commandes en attente. 2. Il confirme la commande après appel à la cliente : le système décrémente le stock de chaque ligne de façon atomique. 3. Il marque la commande <i>prête</i> lorsqu'elle est préparée. 4. Il la marque <i>terminée</i> à la remise : le système crédite les points de fidélité correspondants et met à jour le compteur de visites.
Alternatives	2a. Le stock est devenu insuffisant : la confirmation échoue et la commande reste en attente. *a. Une transition non autorisée est demandée (par exemple <i>terminée</i> depuis <i>en attente</i>) : le système la refuse. *b. La commande est annulée après confirmation : le stock décrémenté est restitué.
Postconditions	La commande est au statut <i>terminée</i> ; les points sont crédités exactement une fois

2.4 Diagrammes de séquence

Cinq processus ont été modélisés sous forme de diagrammes de séquence. Ils ont été choisis non pour leur fréquence d'usage, mais parce que dans chacun d'eux *l'ordre des opérations détermine la correction du résultat* — ce qu'un diagramme de cas d'utilisation ne peut pas montrer.

2.4.1 Inscription et vérification de l'adresse e-mail

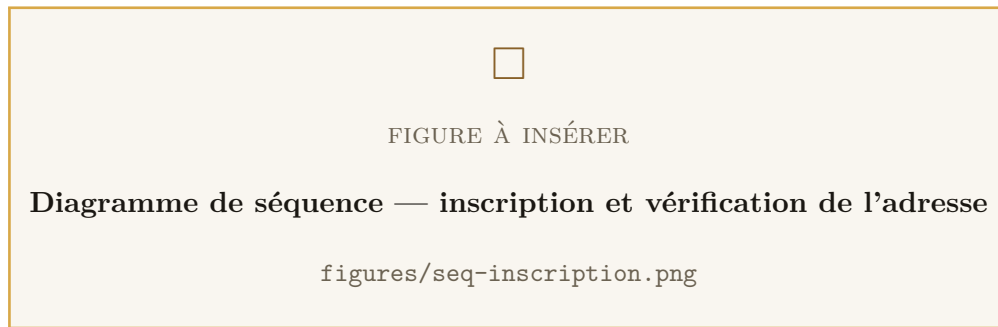


Figure 2.6 : Diagramme de séquence — inscription et vérification de l'adresse

L'inscription crée le compte, son compte de fidélité associé, et renvoie immédiatement les jetons de session. Ce dernier point n'allait pas de soi : une première version renvoyait un simple accusé de création, et l'application enchaînait sur un appel de connexion. Entre les deux appels, la protection anti-force brute pouvait refuser le second, laissant la cliente devant un message d'échec alors même que son compte venait d'être créé. L'inscription renvoie donc désormais exactement ce que renvoie la connexion.

Un code à six chiffres est envoyé par e-mail dans la langue choisie par la cliente. Seule son empreinte est conservée en base : une fuite de la base ne livrerait aucun code exploitable. Le nombre de tentatives est plafonné — six chiffres représentent un million de possibilités, ce qui se parcourt en quelques minutes sans ce garde-fou. L'émission d'un nouveau code efface le précédent du même usage, ce qui garantit qu'au plus une ligne active existe par cliente et par usage.

2.4.2 Réservation d'un créneau

C'est le processus le plus délicat du système, et le diagramme ci-dessous en montre la raison.

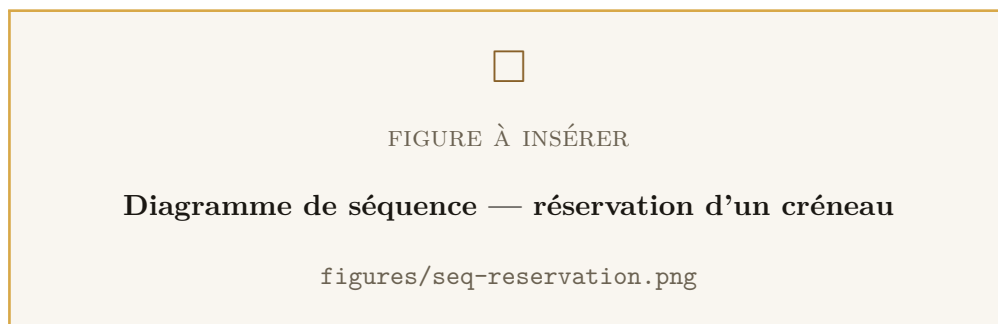


Figure 2.7 : Diagramme de séquence — réservation d'un créneau

La difficulté tient à ce que la contrainte de capacité porte sur un *ensemble* de lignes — le

nombre de rendez-vous qui chevauchent le créneau — et non sur une ligne particulière. Aucune écriture conditionnelle ne peut donc la garantir. La séquence « compter les rendez-vous du créneau, puis insérer si la capacité le permet » est correcte pour une cliente seule, et fausse dès que deux réservations se croisent : les deux comptent avant que l'une ou l'autre n'ait inséré, les deux trouvent de la place, et deux cabines en accueillent trois.

La solution retenue consiste à ouvrir une transaction et à y prendre un verrou consultatif PostgreSQL, tenu jusqu'à la fin de la transaction. La seconde réservation attend que la première soit écrite avant de compter. Ce verrou se libère automatiquement au *commit* comme au *rollback*, ce qui écarte tout risque de blocage résiduel.

À l'intérieur de cette transaction, la vérification enchaîne quatre contrôles : la prestation existe et est active, la date n'est pas passée, le jour n'est ni fermé ni exceptionnellement clos, et la capacité du créneau n'est pas atteinte. Le tarif est enfin figé sur le rendez-vous, afin qu'une révision ultérieure des prix ne modifie pas les points gagnés sur un rendez-vous déjà réservé.

2.4.3 Traitement d'une commande et gestion du stock

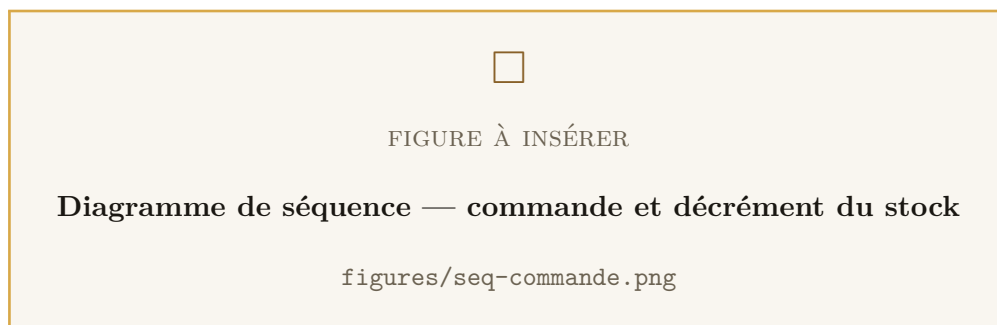


Figure 2.8 : Diagramme de séquence — commande et décrétement du stock

Le décrétement du stock à la confirmation obéit à la même exigence d'atomicité que la réservation, mais il se traite plus simplement : la contrainte porte cette fois sur une ligne unique — le produit — et une mise à jour conditionnelle suffit. La condition « décrétement de n à condition que le stock soit au moins égal à n » est évaluée par la base au moment de l'écriture. Si elle n'est pas satisfaite, aucune ligne n'est affectée et la transaction entière est annulée : la commande reste en attente et le stock demeure cohérent.

2.4.4 Clôture d'un rendez-vous et crédit de fidélité

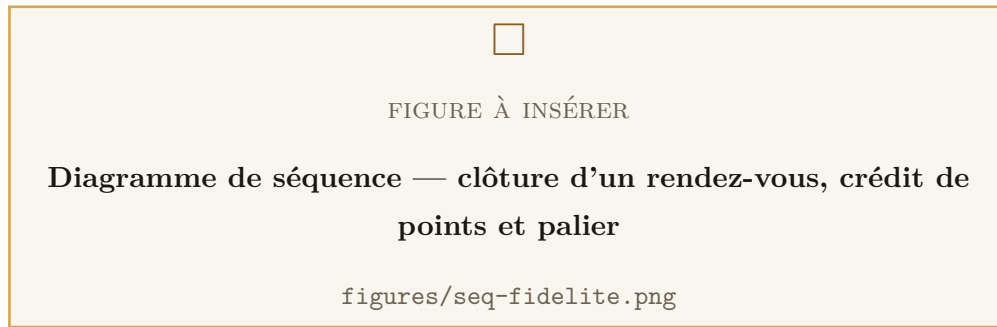


Figure 2.9 : Diagramme de séquence — clôture d'un rendez-vous, crédit de points et palier

La clôture d'un rendez-vous déclenche une chaîne d'opérations qui doivent toutes réussir ou toutes échouer : passage au statut *terminé*, calcul des points sur le tarif figé, incrément du solde et du compteur de visites, écriture du mouvement d'historique, puis évaluation des paliers de visites. Le tout s'exécute dans une seule transaction.

L'idempotence est assurée à deux niveaux. Le mouvement de fidélité porte une référence unique vers le rendez-vous : une clôture rejouée ne peut pas créditer deux fois. Les déblocages de paliers portent une contrainte d'unicité sur le triplet (compte, palier, cycle), où le cycle indique combien de fois le seuil a été franchi — ce qui permet à un palier d'être récurrent tout en gardant chaque déblocage unique.

2.4.5 Rotation des jetons de session

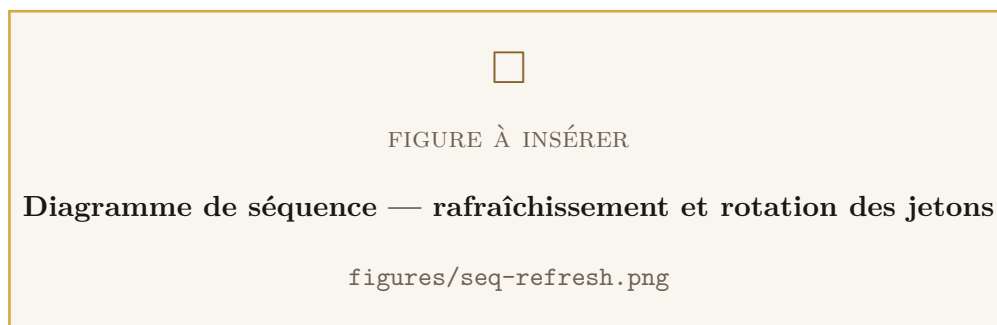


Figure 2.10 : Diagramme de séquence — rafraîchissement et rotation des jetons

Le jeton d'accès est volontairement de courte durée. Pour éviter de demander à la cliente de se reconnecter plusieurs fois par jour, un jeton de rafraîchissement de longue durée permet d'en obtenir un nouveau.

Ce mécanisme est protégé par une rotation : chaque rafraîchissement révoque le jeton présenté et en émet un nouveau, rattaché à la même famille. Si un jeton déjà utilisé

est présenté une seconde fois, c'est le signe qu'il a été dérobé — soit par l'attaquant, soit par la cliente légitime, sans qu'il soit possible de savoir lequel. Le système révoque alors toute la famille : les deux parties sont déconnectées, et l'accès frauduleux prend fin.

2.5 Diagramme de classes

2.5.1 Vue d'ensemble

Le diagramme de classes ci-dessous présente la structure des données persistantes. Il comporte dix-neuf entités, réparties en six domaines : utilisateurs et sessions, catalogues, rendez-vous, commandes, fidélité, et paramétrage du centre.

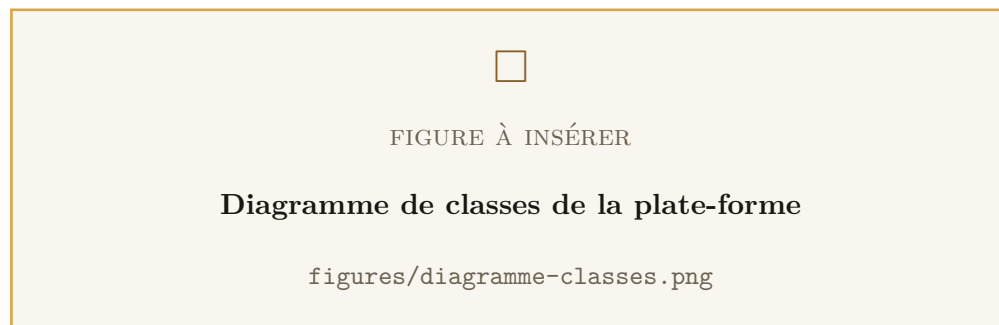


Figure 2.11 : Diagramme de classes de la plate-forme

2.5.2 Choix structurants du modèle

Cinq décisions de modélisation méritent d'être explicitées, car elles ne découlent pas mécaniquement des besoins et constituent l'essentiel de l'apport de conception.

Le figement des montants. Un rendez-vous conserve le tarif de la prestation *tel qu'il était à la réservation*, et chaque ligne de commande conserve le prix unitaire du produit *tel qu'il était à la commande*. Sans cela, une révision des tarifs réécrirait rétroactivement l'historique : une commande passée en juillet afficherait le prix d'août, et les points gagnés sur un rendez-vous changeraient après coup. Le même principe s'applique aux récompenses de palier, dont la référence est figée au déblocage, et aux points dépensés, figés sur le bon émis.

L'anonymisation plutôt que la suppression. Une cliente qui exerce son droit à l'effacement voit son compte vidé de tout ce qui l'identifie — adresse e-mail, nom, téléphone, mot de passe — et marqué comme supprimé, mais sa ligne n'est pas détruite. Ses commandes et ses rendez-vous portent en effet le chiffre d'affaires de l'institut, qui

doit survivre à son départ. Les relations le garantissent structurellement : les rendez-vous et les commandes sont rattachés à la cliente en mode **Restrict**, si bien qu'une suppression échouerait bruyamment plutôt que de détruire l'historique en silence. Un compte ainsi marqué ne peut plus se connecter, ni recevoir d'e-mail, ni réinitialiser son mot de passe.

Le traitement différencié des employées. Les écritures de fidélité saisies par le personnel et les bons honorés au comptoir désignent l'employée concernée, mais en mode **SetNull** et non **Cascade**. Le départ d'une employée ne doit pas effacer les points des clientes qu'elle a servies ni la trace de ce qui leur a été remis : l'écriture demeure, elle ne désigne simplement plus personne.

Le bon de récompense comme entité de première classe. C'est la notion qui manquait à la première version du programme de fidélité. Sans elle, réclamer une récompense offerte la faisait disparaître des deux côtés à la fois — l'application filtrait sur la date de réclamation, la fiche du comptoir aussi — et un échange de points ne laissait qu'une ligne d'historique que personne ne listait. Le bon réunit les deux circuits de récompense en un objet unique : une seule chose à afficher pour la cliente, une seule liste à traiter au comptoir. Son unicité par origine — un déblocage de palier, ou un mouvement de points — constitue le véritable garde-fou d'idempotence : une réclamation rejouée ne peut pas émettre un second bon.

La plage d'ouverture plutôt que le jour ouvré. Les horaires sont modélisés comme des *plages*, et non comme des jours dotés d'une heure d'ouverture et d'une heure de fermeture. Un même jour en porte donc autant que nécessaire — 9 h–13 h puis 14 h–18 h — et un jour sans aucune plage est fermé. Ce modèle exprime naturellement la pause déjeuner, qu'un booléen « fermé » assorti de deux heures ne savait représenter qu'au prix d'une plage trouée. Les fermetures exceptionnelles, elles, sont stockées comme des dates de calendrier et non comme des instants : une fermeture est un jour du calendrier local du centre, et stocker un instant obligerait à choisir une heure arbitraire.

Conclusion

Ce chapitre a présenté l'analyse et la conception du système : les besoins fonctionnels et non fonctionnels, les acteurs, les cas d'utilisation et leur description, les séquences des processus les plus délicats, et la structure des données. Les décisions de modélisation y ont été justifiées plutôt que seulement énoncées, car plusieurs d'entre elles — le figement des montants, l'anonymisation, le bon de récompense — constituent la réponse à des

difficultés qui ne se voyaient qu'à l'usage. Le chapitre suivant expose les choix techniques qui permettent de réaliser cette conception.

CHAPITRE 3

Étude technique

Introduction

Ce chapitre expose les choix techniques du projet. Il présente les langages, les cadrage et le système de gestion de base de données retenus, ainsi que les environnements matériel et logiciel de travail. Il décrit ensuite l'architecture technique de la solution : son découpage en couches, son architecture de déploiement, et la chaîne de sécurité appliquée à chaque requête.

Les choix sont ici justifiés plutôt qu'énumérés. Une technologie ne vaut que par le problème qu'elle résout dans un contexte donné, et plusieurs des options retenues n'auraient pas été les mêmes pour un projet d'une autre nature.

3.1 Technologies de développement

3.1.1 Langages

TypeScript. L'ensemble du code applicatif — API, back-office et application mobile — est écrit en TypeScript, sur-ensemble typé de JavaScript développé par Microsoft. Ce choix unique pour les trois applications a une conséquence directe et précieuse : les types décrivant les données métier sont écrits une fois et compris partout. Le type d'un rendez-vous retourné par l'API est le même côté serveur, côté back-office et côté application mobile.

Le typage statique joue par ailleurs un rôle de filet de sécurité dans un projet qui n'a pas de tests d'interface : le back-office et l'application mobile n'ont pas de suite de tests automatisés, et leur garantie principale est la compilation. Un champ renommé côté serveur produit une erreur de compilation côté client, avant tout déploiement.

SQL. Le langage SQL n'est presque jamais écrit à la main dans le projet — Prisma le génère — à une exception près, précisément là où il est irremplaçable : la prise du verrou consultatif qui sérialise les réservations concurrentes, exprimée par un appel direct à `pg_advisory_xact_lock`. Aucune abstraction ne fournit cette primitive, et c'est elle qui garantit qu'un créneau n'est jamais vendu deux fois.

3.1.2 Cadriciels et bibliothèques

NestJS. L'API repose sur NestJS, cadriciel Node.js structuré autour de modules, de contrôleurs et de services, et bâti sur l'injection de dépendances. Sa structure imposée est un atout dans un projet mené seul : elle évite les choix d'organisation arbitraires et rend chaque module immédiatement localisable. Ses gardes, filtres, intercepteurs et tubes de validation permettent en outre de traiter les préoccupations transversales — droits d'accès, validation, traduction, gestion des erreurs — à un seul endroit, plutôt que répétées dans chaque contrôleur.

Prisma. Prisma assure la correspondance objet-relationnel. Le schéma y est déclaré dans un fichier unique, à partir duquel sont générés à la fois le client typé et les fichiers de migration. Deux propriétés ont pesé dans ce choix : le client est *typé d'après le schéma*, si bien qu'une colonne renommée fait échouer la compilation partout où elle était lue ; et les migrations sont des fichiers SQL versionnés, relus avant application, ce qui rend le passage en production vérifiable.

React et Vite. Le back-office est une application web mono-page écrite en React et servie par Vite. React a été retenu pour la richesse de son écosystème et sa maîtrise préalable ; Vite pour la rapidité de son serveur de développement et la simplicité de sa production statique. L'état global — réduit à la session authentifiée — est géré par Zustand, bibliothèque volontairement minimale, dimensionnée pour ce besoin.

React Native et Expo. L'application mobile est développée avec React Native, sous la chaîne d'outils Expo. Ce choix permet de produire une application Android et iOS à partir d'une base de code unique, tout en partageant le langage et une partie des conventions avec le back-office. Expo apporte en outre les briques natives nécessaires — stockage sécurisé des jetons, polices, images, dégradés, localisation — sans avoir à écrire de code natif.

Bibliothèques notables. Argon2 pour le hachage des mots de passe, choisi pour sa résistance aux attaques matérielles ; `date-fns-tz` pour les conversions entre heure locale du centre et temps universel ; ExcelJS pour la génération des exports ; Node-mailer pour l'acheminement des e-mails ; Helmet pour les en-têtes de sécurité HTTP ; `class-validator` pour la validation déclarative des entrées ; `i18next` pour l'internationalisation de l'application mobile.

3.1.3 Services web

L'API suit le style d'architecture REST, avec JSON pour format d'échange. Ce choix s'imposait pour plusieurs raisons convergentes :

- **L'universalité de HTTP.** Deux clients de natures très différentes — une application native et une application web — consomment la même API sans adaptation.
- **L'absence d'état du protocole.** Chaque requête porte son jeton d'authentification ; aucun état de session n'est conservé côté serveur, ce qui simplifie considérablement le déploiement et la montée en charge.
- **La légèreté du format.** JSON convient particulièrement à un client mobile, dont la bande passante et l'autonomie sont des ressources comptées.
- **L'outillage.** La documentation OpenAPI est générée directement depuis les décorateurs du code, ce qui garantit qu'elle ne diverge pas de l'implémentation.

L'API compte plus de quatre-vingts routes réparties en douze contrôleurs. Le tableau ci-dessous en donne la répartition.

Tableau 3.1 : Répartition des routes de l'API par domaine

Domaine	Routes	Principales opérations
/auth	9	Inscription, connexion, profil courant, vérification d'adresse, renvoi de code, mot de passe oublié, ré-initialisation, rafraîchissement, déconnexion
/users	6	Profil, modification, changement de mot de passe, suppression de compte, liste des utilisateurs, attribution de rôle
/services	10	Catalogue public, catégories, détail, gestion complète (administration)
/products	10	Catalogue public, catégories, détail, gestion complète (administration)
/appointments	7	Disponibilités, réservation, mes rendez-vous, annulation, réservation pour une cliente, changement de statut, report
/orders	6	Commande, mes commandes, détail, annulation, liste (personnel), changement de statut
/loyalty	23	Solde, historique, récompenses, échange, paliers, déblocages, réclamation, bons, gestion (administration), ajout manuel, journal d'audit
/opening-hours	5	Consultation, remplacement des plages, fermetures exceptionnelles
/settings	2	Consultation et modification des réglages du centre
/exports	3	Exports Excel des clientes, commandes et rendez-vous
/uploads	1	Téléversement d'une image de catalogue
/health	1	Sonde de santé pour la supervision et le déploiement

3.1.4 Système de gestion de base de données

PostgreSQL 16 a été retenu comme système de gestion de base de données. Le choix d'un SGBD relationnel ne faisait pas débat — les données du projet sont fortement structurées et fortement liées — mais celui de PostgreSQL plutôt que d'une autre base relationnelle mérite d'être justifié.

Trois de ses propriétés sont directement exploitées :

- **Les verrous consultatifs.** C'est la primitive qui rend correcte la réservation concurrente. Elle n'a pas d'équivalent direct et portable ailleurs, et elle est utilisée au cœur même du moteur de réservation.
- **Les transactions et l'intégrité référentielle.** Le crédit de fidélité, la clôture d'un rendez-vous et le déblocage d'un palier s'exécutent dans une transaction unique. Les modes de suppression différenciés — **Cascade**, **Restrict**, **SetNull** selon la relation — portent une partie des règles de conservation de l'historique dans le schéma lui-même, où aucun oubli applicatif ne peut les contourner.
- **Le type décimal exact.** Les montants sont stockés en `Decimal(10,2)`, et non en nombre flottant. Un montant en virgule flottante accumule des erreurs d'arrondi qui finissent par se voir sur un total.

Le schéma comporte dix-neuf tables et a connu vingt migrations versionnées depuis l'initialisation du projet.

3.1.5 Outils de développement

3.1.5.1 Environnement matériel

Tableau 3.2 : Environnement matériel de développement

Élément	Caractéristiques
Poste de développement	Ordinateur portable, processeur multicœur, 16 Go de mémoire vive, disque SSD
Système d'exploitation	Windows avec sous-système Linux, et Linux
Terminal de test mobile	Téléphone Android physique et émulateur Android
Serveur de base de données	Conteneur Docker PostgreSQL 16, exécuté localement

Un piège rencontré. Sous Windows, la plage de ports 2933–3032 est réservée par Hyper-V et par le sous-système Linux. Le port 3000, choisi par défaut par la plupart des projets Node.js, y produit une erreur d'accès refusé qui ne désigne pas sa cause. Le port d'écoute de l'API a donc été rendu configurable par variable d'environnement — ce qui s'est révélé nécessaire également au déploiement, la plupart des hébergeurs imposant le leur.

3.1.5.2 Environnement logiciel

Tableau 3.3 : Environnement logiciel de développement

Outil	Usage
Visual Studio Code	Éditeur de code des trois applications
Node.js 22	Environnement d'exécution JavaScript côté serveur et outillage
Docker et Docker Compose	Base de données locale, puis conteneurisation de l'API pour le déploiement
Git et GitHub	Gestion de versions et hébergement du dépôt
GitHub Actions	Intégration continue : compilation et tests à chaque modification
Prisma Studio	Inspection et correction ponctuelle des données en développement
Swagger UI	Documentation interactive de l'API, générée depuis le code
Jest et Supertest	Tests unitaires, tests d'intégration et tests de concurrence
ESLint et Prettier	Analyse statique et mise en forme homogène du code
Expo Go	Exécution de l'application mobile sur un téléphone réel sans compilation native
Lucidchart	Réalisation des diagrammes UML

3.2 Architecture technique

3.2.1 Architecture en couches

Une vue d'ensemble permet de dégager les couches du système. L'application étant à la fois web et mobile, l'architecture retenue est une architecture en quatre niveaux : deux clients distincts, un serveur d'application et un serveur de données.

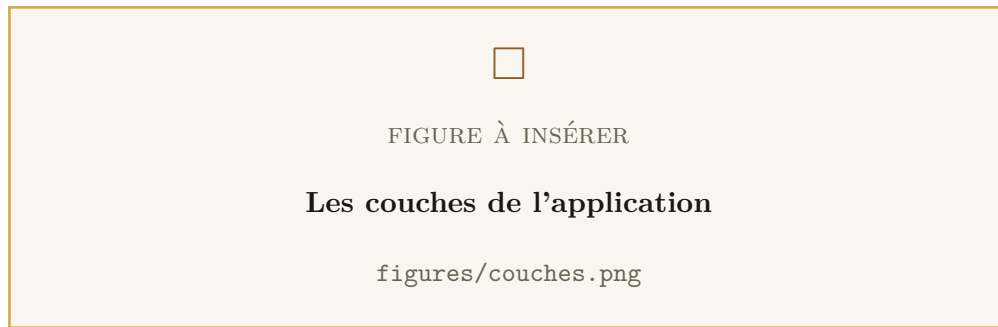


Figure 3.1 : Les couches de l'application

- **La couche de présentation.** Elle encapsule l'interface utilisateur. Elle est double : l'application mobile pour les clientes, le back-office web pour l'institut. Elle n'implémente *aucune* règle métier : elle affiche ce que le serveur lui transmet et lui soumet ce que l'utilisatrice saisit. Les contrôles qu'elle effectue — un champ obligatoire, un format d'adresse — servent uniquement le confort de saisie et sont systématiquement rejoués côté serveur.
- **La couche métier.** C'est le serveur d'application NestJS. Elle porte l'intégralité des règles : moteur de créneaux, machines à états, calcul des points, contrôle des droits, validation des entrées. Elle expose ses services sous forme d'API REST.
- **La couche d'accès aux données.** Assurée par Prisma, elle traduit les opérations métier en requêtes SQL, gère les transactions et garantit le typage des résultats.
- **La couche de persistance.** La base PostgreSQL. Elle ne se contente pas de stocker : elle porte des règles d'intégrité — unicité, clés étrangères, modes de suppression — et fournit les primitives de verrouillage qui rendent la concurrence correcte.

3.2.2 Architecture de déploiement

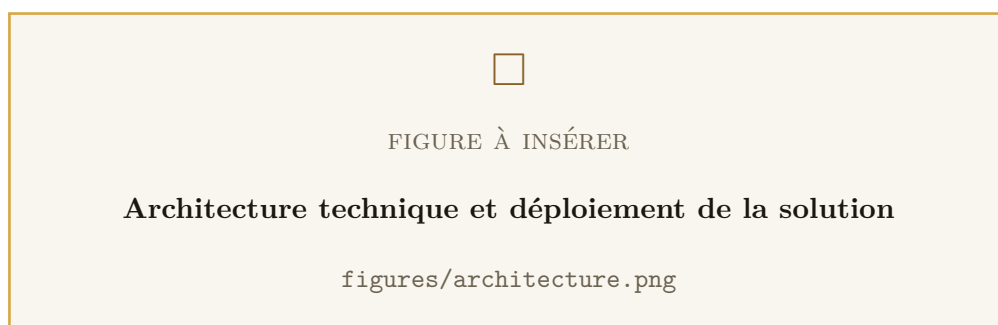


Figure 3.2 : Architecture technique et déploiement de la solution

Le déploiement repose sur Docker Compose et enchaîne trois services dans un ordre imposé :

1. **PostgreSQL**, doté d'un contrôle de santé et d'un volume persistant. Son port n'est publié que sur l'interface locale.
2. **Un service de migration**, à usage unique : il applique les migrations en attente, puis s'arrête. Il est délibérément séparé de l'API, afin qu'une migration en échec se présente comme une étape ratée assortie de son message, et non comme une application qui redémarre en boucle sans raison apparente. Il utilise la commande `migrate deploy`, qui applique ce qui manque et refuse de toucher à l'existant — et jamais `migrate dev`, qui peut réinitialiser la base.
3. **L'API**, qui ne démarre qu'une fois les migrations appliquées avec succès.

L'image de l'API est construite en trois étapes, afin que l'image livrée ne contienne que ce qui sert à servir — ni compilateur, ni cadriciel de test, ni outillage de développement. Le résultat mesuré est de 178 Mo.

Une décision de déploiement. L'API est publiée sur l'interface locale uniquement. C'est délibéré : un reverse proxy (Caddy ou nginx) doit se placer devant elle pour assurer le chiffrement HTTPS. Publier le port directement reviendrait à servir l'API en clair, jetons d'authentification compris. Le jour où ce proxy est mis en place, un réglage devient indispensable : la limitation de débit compte par adresse IP, et derrière un proxy toutes les clientes partagent la sienne. Sans ce réglage, l'institut entier se retrouve sur un compteur unique et l'API commence à refuser des requêtes au hasard dès qu'il y a du monde — un symptôme déroutant, parfait en test avec une seule personne, et que rien dans les journaux ne rattache à sa cause.

3.2.3 Place de la sécurité dans l'architecture

L'architecture ci-dessus porte une décision qui relève autant de la sécurité que de la conception : *toute requête traverse une chaîne de filtres avant d'atteindre la moindre ligne de logique métier*. En-têtes de sécurité, limitation de débit, contrôle d'origine, authentification, autorisation par rôle et validation stricte des entrées s'appliquent successivement, et un filtre d'exceptions global intercepte tout ce qui remonte en sens inverse.

Ces mécanismes ne sont pas des ajouts posés sur l'architecture : ils en font partie, au même titre que le découpage en couches. C'est la raison pour laquelle ils ne sont pas détaillés ici mais font l'objet du chapitre 4, qui leur est entièrement consacré — du

modèle de menaces retenu jusqu'aux limites qui subsistent.

Conclusion

Ce chapitre a présenté les choix techniques du projet et leur justification : un langage unique pour les trois applications, un cadrage structurant pour l'API, un ORM typé adossé à PostgreSQL, et une architecture en quatre niveaux où la couche métier est seule dépositaire des règles. L'architecture de déploiement a été exposée, ainsi que la place qu'y tient la chaîne de traitement de sécurité. Le chapitre suivant est consacré à cette dernière : il présente le modèle de menaces retenu et les contre-mesures qui en découlent.

CHAPITRE 4

Sécurité de la plate-forme

Introduction

Ce chapitre est consacré à la sécurité de la solution. Il occupe une place particulière dans ce rapport, et pour deux raisons.

La première tient à la nature du système. La plate-forme traite des données personnelles — identité, coordonnées, historique de fréquentation d'un institut de beauté — et porte des opérations à valeur économique : réservations, commandes, points de fidélité convertibles en prestations gratuites. Chacune de ces trois catégories appelle des garanties différentes, et aucune n'est satisfaite par défaut.

La seconde tient au fait qu'un système exposé sur Internet n'a pas seulement des utilisatrices : il a des visiteurs. L'API est publique par construction — l'application mobile l'appelle depuis les téléphones de clientes, sur des réseaux quelconques. Tout ce qui n'est pas explicitement refusé côté serveur est, en pratique, autorisé.

La démarche suivie n'a donc pas consisté à ajouter de la sécurité à un système achevé, mais à traiter chaque décision de conception comme portant une part de sécurité. Ce chapitre restitue cette démarche : le modèle de menaces retenu, les principes directeurs, puis les contre-mesures effectivement implémentées, domaine par domaine. Il se termine par une synthèse et par un inventaire honnête des limites qui subsistent.

4.1 Démarche et modèle de menaces

4.1.1 Surface d'attaque

La première étape a consisté à délimiter la surface exposée et à situer les frontières de confiance du système.

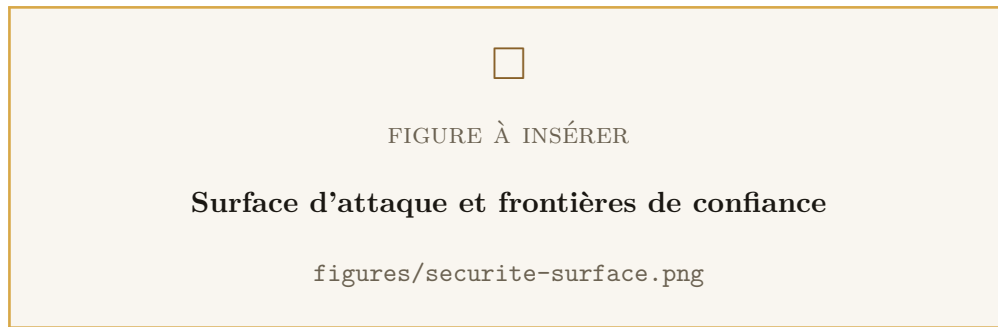


Figure 4.1 : Surface d'attaque et frontières de confiance

Cinq points d'entrée existent, et un seul est réellement de confiance :

- **L'API HTTP**, atteignable par quiconque connaît son adresse. Sa documentation OpenAPI décrit d'ailleurs chacune de ses routes — raison pour laquelle elle est coupée en production.
- **L'application mobile**, qui s'exécute sur le téléphone de la cliente. Elle échappe totalement à notre contrôle : son code est lisible, son trafic interceptable, ses contrôles contournables. *Aucune règle de sécurité ne peut y résider.*
- **Le back-office web**, dans le même cas : c'est du JavaScript servi au navigateur. Masquer un bouton d'administration n'a jamais protégé la route correspondante.
- **Les images téléversées**, seuls fichiers que l'application écrit sur le disque du serveur — et qu'elle republie ensuite en HTTP.
- **La base de données**, qui n'est pas exposée : son port n'est publié que sur l'interface locale du serveur.

Il en découle un principe qui structure tout ce chapitre : *la frontière de confiance passe à l'entrée de l'API, et nulle part ailleurs*. Tout ce qui arrive d'un client — corps de requête, en-têtes, type MIME déclaré, extension de fichier, rôle affiché dans l'interface — est une donnée hostile jusqu'à vérification par le serveur.

4.1.2 Modèle de menaces

Les menaces ont été recensées selon les six catégories du modèle STRIDE, en retenant pour chacune les scénarios réalistes pour ce système.

Tableau 4.1 : Modèle de menaces STRIDE appliqué à la plate-forme

Catégorie	Scénarios retenus
Usurpation (<i>Spoofing</i>)	Force brute sur le formulaire de connexion ; vol d'un jeton de session ; devinette d'un code de réinitialisation à six chiffres ; forge d'un jeton d'administration si le secret de signature est faible.
Altération (<i>Tampering</i>)	Injection SQL ; modification du rôle porté dans la requête ; téléversement d'un fichier exécutable déguisé en image ; falsification de l'en-tête d'adresse IP réelle.
Répudiation	Ajout manuel de points de fidélité sans trace ; remise d'un bon de récompense sans enregistrement de qui l'a honoré.
Divulgation (<i>Information disclosure</i>)	Lecture des commandes ou des rendez-vous d'une autre cliente ; fuite du schéma de base par les messages d'erreur ; énumération des comptes existants ; export massif de la base clientes.
Déni de service	Saturation de l'API par un client unique ; épuisement du processeur par le hachage de mots de passe ; export d'un volume de lignes qui bloque le serveur ; réservation en masse de créneaux.
Élévation de privilège	Accès direct à une route d'administration depuis un compte cliente ; promotion de son propre rôle ; usage d'un jeton émis avant la suppression du compte.

4.1.3 Principes directeurs

Quatre principes ont guidé les décisions.

Le serveur fait autorité. Toute règle de sécurité est appliquée par l'API. Les interfaces peuvent masquer, désactiver ou pré-valider : ce n'est jamais qu'un confort d'usage, jamais une protection.

Défense en profondeur. Aucune contre-mesure unique n'est réputée suffisante. Un compte supprimé est ainsi protégé par trois verrous indépendants : le refus de connexion sur son marqueur de suppression, le remplacement de son mot de passe par le haché d'un secret que personne ne connaît, et le rejet, à chaque requête, des jetons déjà émis.

Sécurité par défaut, et échec fermé. Toute route est authentifiée et comptée sauf déclaration contraire explicite. Toute propriété non déclarée dans le contrat d'une route fait rejeter la requête, au lieu d'être ignorée en silence. Et lorsqu'une configuration est

incohérente, l'API *refuse de démarrer* plutôt que de fonctionner dans un état dégradé que rien ne signalerait.

Moindre privilège. Trois rôles seulement, chacun strictement borné. L'image de production ne contient aucun compilateur et le processus n'y tourne pas en tant que `root`. Le port de la base de données n'est pas publié.

4.2 Authentification

4.2.1 Stockage des mots de passe

Les mots de passe sont hachés avec **Argon2id**, lauréat du *Password Hashing Competition* et normalisé par la RFC 9106. Le choix s'est porté sur lui plutôt que sur `bcrypt` pour sa résistance aux attaques matérielles : Argon2 est conçu pour être coûteux en *mémoire* autant qu'en temps de calcul, ce qui ôte aux cartes graphiques et aux circuits dédiés l'avantage massif dont ils disposent face aux fonctions purement calculatoires.

Aucun mot de passe n'est stocké, journalisé ou renvoyé en clair, à aucun moment. La politique appliquée impose au minimum huit caractères, dont une majuscule et une minuscule, et elle est *la même* à l'inscription, au changement de mot de passe et à la réinitialisation — une exigence relâchée sur l'un de ces trois chemins annulerait les deux autres.

4.2.2 Jetons d'accès

L'authentification des requêtes repose sur des jetons JWT signés, de durée volontairement courte : **quinze minutes**. Cette brièveté est la réponse au fait qu'un JWT est, par nature, invérifiable auprès d'une liste de révocation sans requête supplémentaire : elle borne la fenêtre pendant laquelle un jeton dérobé reste exploitable.

Le secret de signature est lu par une méthode qui *lève une exception s'il est absent*, aussi bien à la signature qu'à la vérification. C'est délibéré : une valeur de repli en dur — le réflexe habituel pour « que ça démarre en développement » — rendrait tous les jetons forgeables par quiconque a lu le dépôt.

Le jeton n'est cependant pas cru sur parole. À chaque requête, la stratégie de vérification recharge le compte en base et refuse le jeton si le compte n'existe plus ou a été anonymisé.

```
async validate(payload: { sub: string; email: string; role: string }) {
  const user = await this.userService.findById(payload.sub);
  // Un compte anonymise est traite comme inexistant. Sans ce test, l'accès
  // token emis juste avant la suppression continuerait d'ouvrir le compte
  // jusqu'à son expiration.
```

```
if (!user || user.deletedAt) {  
    throw new UnauthorizedException();  
}  
return { id: user.id, email: user.email, role: user.role };  
}
```

Listing 4.1 : Vérification du jeton d'accès : la signature ne suffit pas

Ce rechargement a un second effet, moins évident et tout aussi important : le rôle appliqué est celui *de la base*, et non celui inscrit dans le jeton. Une rétrogradation prend donc effet immédiatement, sans attendre l'expiration des jetons en circulation.

4.2.3 Jetons de rafraîchissement, rotation et détection de vol

Des jetons d'accès de quinze minutes obligeraient la cliente à se reconnecter plusieurs fois par jour. Un jeton de rafraîchissement de longue durée résout ce problème — et en crée un autre : c'est désormais lui qui vaut la session, et sa durée de vie le rend d'autant plus intéressant à dérober.

Trois mesures répondent à ce risque.

- **Il n'est pas stocké en clair.** La base ne contient que son empreinte SHA-256. Une lecture de la base ne livre donc aucune session utilisable. Le hachage rapide suffit ici, contrairement au cas des mots de passe : le jeton est une valeur aléatoire de 48 octets, et non un secret choisi par un être humain — il n'existe pas de dictionnaire de jetons.
- **Il tourne.** Chaque rafraîchissement révoque le jeton présenté et en émet un nouveau, rattaché à la même *famille*.
- **Sa réutilisation est détectée.** Présenter deux fois le même jeton est impossible en usage normal. C'est en revanche exactement ce qui se produit lorsqu'un jeton a été dérobé : l'attaquant et la cliente légitime l'utilisent chacun de leur côté. Impossible de savoir lequel des deux se présente — le système révoque donc *toute la famille*. Les deux sont déconnectés, et l'accès frauduleux prend fin. Le coût pour la cliente est une reconnexion ; le bénéfice est la fin d'une session volée qui, sans cela, se prolongerait indéfiniment par rotations successives.

Côté mobile, le jeton n'est pas rangé dans le stockage ordinaire de l'application mais dans le **coffre sécurisé du système d'exploitation** — trousseau iOS, *Keystore* Android — adossé au matériel du téléphone.

Le déroulement complet, rotation et détection de réutilisation comprises, est donné par la figure 2.10 du chapitre 2.

4.2.4 Codes à usage unique

La vérification d'adresse et la réinitialisation de mot de passe reposent sur un code à six chiffres envoyé par courriel. Un tel code est court par nécessité — il doit être recopié à la main — donc faible par construction : un million de possibilités se parcourent en quelques minutes. Cinq mesures le rendent néanmoins sûr.

Tableau 4.2 : Protection des codes à usage unique

Mesure	Ce qu'elle empêche
Génération par <code>randomInt</code>	Générateur cryptographique, et non <code>Math.random</code> dont la suite est prédictible : un code observé ne permet pas de deviner le suivant.
Stockage haché	Une lecture de la base ne livre aucun code exploitable.
Plafond de cinq tentatives	Ferme la porte à l'énumération : sans lui, six chiffres se devinent.
Validité de quinze minutes	Borne la fenêtre d'exploitation d'un code intercepté.
Comparaison en temps constant	<code>timingSafeEqual</code> : le temps de réponse ne trahit pas le nombre de chiffres corrects, ce qui interdit de reconstituer le code position par position.

À quoi s'ajoute une propriété structurelle : la recherche s'effectue par *couple* (*compte, usage*) et jamais par le code seul. Un code deviné ne vaut donc que pour le compte auquel il a été émis — alors qu'une recherche par code seul aurait fait qu'un code trouvé au hasard ouvre n'importe quel compte l'ayant reçu, ce qui multiplie les chances de l'attaquant par le nombre de codes en circulation. Enfin, émettre un nouveau code efface le précédent du même usage : au plus un code actif par cliente et par usage.

4.2.5 Défense contre l'énumération de comptes

Savoir quelles adresses possèdent un compte est une information exploitable : elle alimente le harponnage ciblé et, s'agissant d'un institut de beauté, elle révèle une information personnelle en soi.

Deux canaux la trahissent, et tous deux ont été fermés.

Le canal des messages. La demande de réinitialisation répond la même chose que l'adresse existe ou non.

Le canal du temps. Ce canal est plus subtil, et c'est précisément ce qui le rend dangereux : il subsiste alors même que les messages sont identiques. Vérifier un mot de passe avec Argon2 coûte plusieurs centaines de millisecondes. Si le serveur répondait immédiatement pour une adresse inconnue — puisqu'il n'a alors aucun haché à comparer — et après ce calcul pour une adresse connue, la seule mesure du délai de réponse suffirait à trier les adresses.

La connexion exécute donc, lorsque l'adresse est inconnue, une vérification *factice* contre le haché d'une valeur aléatoire. Elle échoue toujours : seul le temps consommé compte.

```
// Hash de reference, calcule une seule fois, sur une valeur aleatoire que
// personne ne connait. Il ne protege rien : il sert uniquement a faire
// travailler argon2 aussi longtemps qu'une vraie verification, pour qu'un
// email inconnu reponde au meme rythme.
private async burnPasswordComparison(candidate: string): Promise<void> {
  this.dummyHashPromise ??= argon2.hash(randomBytes(32).toString('hex'));
  const dummyHash = await this.dummyHashPromise;
  await argon2.verify(dummyHash, candidate).catch(() => false);
}
```

Listing 4.2 : Uniformisation du temps de réponse à la connexion

4.3 Autorisation et contrôle d'accès

4.3.1 Modèle de contrôle d'accès

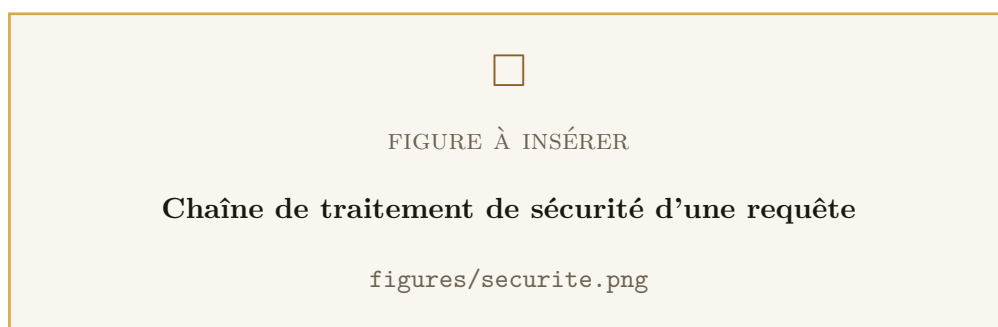
Le contrôle d'accès repose sur un modèle par rôles (RBAC) à trois niveaux strictement ordonnés. Le tableau ci-dessous donne la matrice des droits effectivement appliquée par le serveur.

Tableau 4.3 : Matrice des droits par rôle

Opération	Public	Cliente	Personnel	Gérante
Consulter les catalogues actifs	✓	✓	✓	✓
Réserver, commander pour soi		✓	✓	✓
Consulter <i>ses</i> rendez-vous et commandes		✓	✓	✓
Consulter ceux de <i>toutes</i>			✓	✓
Réserver pour le compte d'une cliente			✓	✓
Changer un statut, honorer un bon			✓	✓
Créditer des points à la main			✓	✓
Gérer le catalogue et les stocks				✓
Régler horaires, capacité, fermetures				✓
Définir récompenses et paliers				✓
Gérer les comptes et les rôles				✓
Consulter le journal d'audit				✓
Exporter les données (Excel)				✓
Téléverser une image				✓

4.3.2 Application par gardes successifs

Cette matrice n'est pas une convention documentaire : elle est appliquée par deux gardes que toute requête traverse, au sein d'une chaîne de filtres qu'elle parcourt *avant* d'atteindre la moindre ligne de logique métier.

**Figure 4.2** : Chaîne de traitement de sécurité d'une requête

Le premier vérifie la signature et la validité du jeton. Il est actif par défaut : une route n'échappe à l'authentification que si elle est *explicitement* déclarée publique. Ce sens du défaut est essentiel — avec la convention inverse, une route ajoutée un jour de fatigue serait ouverte, et rien ne le signalerait.

Le second compare le rôle du compte aux rôles exigés par la route. Il est indépendant de tout ce que l'interface affiche ou masque : une route d'administration atteinte par un autre moyen — l'outil de développement du navigateur, un client HTTP en ligne de commande, la documentation OpenAPI — est refusée par le serveur lui-même.

4.3.3 Contrôle de propriété des ressources

Le rôle ne suffit pas. Deux clientes portent le même rôle, et rien dans ce rôle n'empêche l'une de demander la commande de l'autre : c'est la faille de type *Insecure Direct Object Reference*, l'une des plus répandues et des plus faciles à exploiter — il suffit de changer un identifiant dans une adresse.

Chaque route qui expose une ressource nominative vérifie donc, en plus du rôle, que la ressource appartient bien au demandeur. La vérification est faite *à la lecture en base*, en filtrant sur le propriétaire, et non après coup : une commande qui n'appartient pas à la cliente est traitée comme introuvable, ce qui évite au passage de confirmer son existence.

Les identifiants sont par ailleurs des UUID et non des entiers séquentiels. Ce n'est pas une protection en soi — la vérification de propriété reste la seule qui compte — mais cela ôte à un attaquant la possibilité d'énumérer les ressources en incrémentant un nombre, et empêche de déduire le volume d'activité de l'institut d'un identifiant observé.

4.3.4 Invariants d'administration

Deux invariants protègent le compte de direction, et ils sont appliqués par le serveur :

- **une seule administratrice à la fois** — promouvoir un compte exige de vérifier qu'aucun autre ne porte déjà ce rôle ;
- **jamais zéro administratrice** — rétrograder la dernière est refusé. Sans ce garde-fou, une fausse manipulation rendrait la gestion du centre définitivement inaccessible, sans autre recours qu'une intervention directe en base.

4.4 Sécurité des entrées et des sorties

4.4.1 Validation stricte des entrées

Un tube de validation global s'applique à toutes les routes, avec trois réglages qui comptent :

- **liste blanche** — seules les propriétés déclarées dans le contrat de la route sont conservées ;
- **rejet des propriétés inconnues** — une propriété non déclarée fait *échouer* la requête au lieu d'être silencieusement ignorée. C'est ce réglage qui ferme la porte à l'affectation en masse : soumettre un champ `role` ou `pointsBalance` dans une mise à jour de profil est rejeté, et non toléré ;
- **conversion typée** — les valeurs sont converties dans le type attendu avant d'atteindre la logique métier, qui ne reçoit donc jamais une chaîne là où elle attend un nombre ou une date.

Les règles de validation sont déclarées au plus près des données, dans les objets de transfert, et sont partagées : la politique de mot de passe, le format des codes à six chiffres et celui des numéros de téléphone marocains sont des validateurs uniques, réutilisés partout où la donnée apparaît.

4.4.2 Injection SQL

Toutes les requêtes passent par l'ORM, qui les émet sous forme paramétrée : les valeurs ne sont jamais concaténées dans le texte de la requête.

Le projet compte *une* requête SQL écrite à la main — la prise du verrou de réservation. Elle est écrite avec un gabarit étiqueté (*tagged template*), lui aussi paramétré, et sa seule valeur variable est une constante du code, jamais une entrée utilisateur.

4.4.3 Contrôle des fichiers téléversés

Le téléversement d'images est le seul chemin par lequel un utilisateur écrit un fichier sur le serveur — fichier ensuite republié en HTTP. C'est donc le point le plus sensible du système, et il est traité comme tel.

- **Réservé à la gérante.** Aucune cliente ne peut téléverser quoi que ce soit.
- **Reconnaissance par signature binaire.** Le type déclaré par le navigateur et l'extension du nom de fichier viennent tous deux du client et se falsifient en une ligne. Seuls les premiers octets du fichier disent ce qu'il *est* : trois signatures sont acceptées — JPEG, PNG, WebP — et c'est la signature reconnue, jamais l'extension fournie, qui détermine l'extension enregistrée.

- **Le SVG est refusé.** C'est un refus délibéré, et non un oubli : le SVG est du XML susceptible de contenir du JavaScript. Servi depuis le domaine de l'application, il s'exécuterait avec les droits de celle-ci — une faille XSS stockée, ouverte par un simple téléversement d'image.
- **Nom de fichier tiré au sort.** Le nom fourni par le client est jeté ; le fichier est enregistré sous un identifiant aléatoire. Cela ferme la traversée de répertoire (un nom du type `../../etc/passwd`) et l'écrasement d'un fichier existant.
- **Taille plafonnée à 5 Mo,** et un seul fichier par requête.
- **Cinq minutes de recul.** Les images sont servies avec un cache d'un an, marqué immuable — ce qui est sûr précisément parce que le nom est tiré au sort : remplacer une photo crée un nouveau nom, jamais une nouvelle version du même. Il n'y a donc rien à invalider.

4.4.4 Fuite d'information par les messages d'erreur

Un message d'erreur technique renseigne un attaquant : noms de tables, noms de colonnes, noms de contraintes, parfois valeurs. Un filtre global intercepte donc toute exception avant qu'elle n'atteigne le client et applique trois traitements :

1. les exceptions métier levées volontairement passent intactes — leur message est destiné à l'utilisatrice ;
2. les erreurs de base de données identifiées sont traduites en langage métier ;
3. **tout le reste devient un message neutre**, le détail et la pile d'appels ne vivant que dans les journaux du serveur.

Les journaux eux-mêmes ont été traités comme un risque : une erreur serveur y consigne l'*identifiant* de l'utilisateur, jamais son adresse électronique — assez pour rejouer l'incident, pas assez pour reconstituer un fichier d'adresses à partir des journaux.

4.4.5 Injection d'en-tête de courriel

Le nom de l'expéditeur est inséré entre guillemets dans un en-tête **From**. Un retour à la ligne y couperait l'en-tête en deux et permettrait d'en injecter d'autres — un **Bcc** vers une adresse tierce, par exemple. Les caractères de rupture sont donc retirés avant insertion. La valeur vient certes de la configuration et non d'un client, mais un fichier d'environnement recopié de travers suffirait à produire l'effet, et l'échec serait silencieux.

4.5 Sécurité du transport et des en-têtes

4.5.1 Chiffrement du transport

L'API n'est publiée que sur l'interface locale du serveur. Ce choix est une mesure de sécurité et non une limitation : il rend *impossible* de la joindre sans passer par le reverse proxy placé devant, à qui revient la terminaison HTTPS. Publier le port directement reviendrait à servir en clair les jetons d'authentification, sur des réseaux Wi-Fi publics compris.

Côté mobile, un garde-fou interdit de configurer une adresse d'API non chiffrée en dehors du développement local.

4.5.2 En-têtes de sécurité HTTP

Helmet retire l'en-tête qui annonçait la technologie du serveur — une information sans usage légitime, qui oriente le choix des exploits — et pose une douzaine d'en-têtes que le navigateur applique.

Tableau 4.4 : Principaux en-têtes de sécurité posés

En-tête	Rôle
Strict-Transport-Security	Impose HTTPS pour les visites suivantes, et ferme la fenêtre de rétrogradation vers HTTP.
Content-Security-Policy	Restreint les sources de scripts : première barrière contre le XSS.
X-Content-Type-Options	Interdit au navigateur de deviner le type d'une ressource, donc d'exécuter comme script un fichier qui n'en est pas un.
X-Frame-Options	Interdit l'inclusion dans un cadre : protection contre le détournement de clic.
Cross-Origin-Resource-Policy	Contrôle quelles origines peuvent charger une ressource.

Un arbitrage assumé. La politique de sécurité du contenu interdit par défaut les scripts en ligne. Or l'interface de documentation OpenAPI s'initialise justement par un script en ligne : l'activer telle quelle afficherait une page blanche. Elle est donc désactivée *uniquement là où cette documentation tourne*, c'est-à-dire en développement. En production, la documentation est coupée et la politique

s'applique pleinement. L'arbitrage porte ainsi sur un environnement qui n'est pas exposé, et non sur le serveur en ligne.

4.5.3 Contrôle des origines

La politique CORS n'autorise que les adresses déclarées du back-office. Un site tiers ne peut donc pas faire émettre à un navigateur des requêtes authentifiées vers l'API.

Un piège de déploiement. Cette liste doit contenir l'adresse réelle du back-office déployé. Non renseignée, elle retombe sur l'adresse de développement : le back-office ne peut plus joindre l'API, et l'échec ne se manifeste que par un blocage dans la console du navigateur — *jamais* par un message du serveur. On cherche donc longtemps du mauvais côté.

4.6 Disponibilité et résistance à l'abus

4.6.1 Limitation du débit

Un compteur par adresse IP plafonne l'ensemble de l'API à 120 requêtes par minute. Les routes coûteuses — celles qui déclenchent un hachage Argon2 ou un envoi de courriel — portent en plus une limite stricte.

Tableau 4.5 : Limites de débit par catégorie de route

Route	Limite	Ce qu'elle protège
Ensemble de l'API	120 / min	Saturation générale du service
Connexion, inscription	10 / min	Force brute sur les mots de passe ; épuisement du processeur par Argon2
Changement de mot de passe	10 / min	Force brute sur le mot de passe actuel
Vérification d'adresse	10 / min	Énumération du code à six chiffres
Renvoi de code	5 / min	Usage de l'API comme relais d'envoi de courriels
Mot de passe oublié	5 / 5 min	Harcèlement par courriel d'une adresse tierce
Téléversement d'image	30 / min	Saturation du disque du serveur

Le réglage qui décide de tout. Le compteur s'appuie sur l'adresse IP du demandeur. Derrière un reverse proxy, cette adresse est celle *du proxy*, identique pour tout le monde : sans le réglage correspondant, l'institut entier partage un unique compteur de 120 requêtes par minute, et l'API se met à répondre 429 au hasard dès qu'il y a du monde. Le symptôme est déroutant — parfait en test avec une seule personne, cassé en production — et rien dans les journaux ne le rattache à sa cause.

Le réglage inverse est tout aussi dangereux, et c'est ce qui en fait un vrai arbitrage : activé sans proxy devant, l'en-tête d'adresse réelle devient falsifiable par n'importe qui, et il suffit alors d'en changer la valeur à chaque requête pour contourner entièrement la limitation. Ce réglage n'est donc à activer *que* si un proxy de confiance est réellement en place.

4.6.2 Plafonnement des volumes

Deux mesures évitent qu'une requête légitime ne dégénère en déni de service.

La pagination obligatoire. Toute liste susceptible de croître — historique de fidélité, commandes, rendez-vous, utilisateurs — est paginée côté serveur. Sans cela, une cliente fidèle depuis trois ans rechargeait la totalité de son historique à chaque ouverture de l'écran.

Le plafond d'export. Les exports Excel comptent les lignes *avant* de charger quoi que ce soit et refusent au-delà de 20 000 lignes, en invitant à restreindre la période. Un export non borné aurait constitué l'opération la plus lourde de l'API, accessible d'une seule requête. Le comptage préalable importe autant que le plafond : c'est lui qui garantit que le refus ne coûte rien.

4.6.3 Correction sous concurrence

Les mécanismes décrits au chapitre 2 — verrou consultatif sur les réservations, écritures conditionnelles atomiques sur le stock et sur le solde de points — relèvent aussi de la sécurité, et pas seulement de la correction fonctionnelle.

Une condition de course est en effet exploitable dès lors qu'elle est déclenchable à volonté. Le débit conditionnel du solde de points en est l'illustration : lire le solde puis le décrémenter permettrait, en lançant plusieurs échanges simultanés, de dépenser plusieurs fois les mêmes points et d'obtenir des récompenses non acquises. Le débit est donc exprimé comme une *écriture conditionnelle* — retirer n points à condition qu'il y

en ait au moins n — évaluée par la base au moment de l'écriture. Si la condition n'est pas satisfaite, aucune ligne n'est modifiée et rien n'est émis.

4.7 Traçabilité et non-répudiation

Certaines opérations ont une valeur économique et doivent laisser une trace opposable.

- **L'ajout manuel de points** exige un motif et enregistre l'employée qui l'a saisi. Un journal d'audit, consultable par la seule gérante, liste ces écritures.
- **La remise d'un bon** enregistre la date et l'identité de la personne qui l'a honoré.
- **Le lien vers l'employée n'est jamais supprimé en cascade.** Le départ d'une employée ne doit ni effacer les points des clientes qu'elle a servies, ni la trace de ce qui leur a été remis : l'écriture demeure, elle ne désigne simplement plus personne.
- **Les montants sont figés** au moment de l'acte. Une révision de tarif ne réécrit pas rétroactivement ce qui a été facturé, ni les points gagnés.

4.8 Protection des données personnelles

4.8.1 Cadre applicable

Le traitement de données personnelles au Maroc est régi par la **loi 09-08**, dont la Commission Nationale de contrôle de la protection des Données à caractère Personnel (CNDP) assure le contrôle. Trois obligations ont eu un effet direct sur la conception.

La minimisation. Seules sont collectées les données nécessaires : identité, adresse électronique, téléphone — ce dernier parce qu'un institut doit pouvoir rappeler une cliente pour confirmer une commande. Aucune donnée de paiement n'est traitée : le règlement s'effectue à la remise, en espèces, et l'application n'en voit rien. C'est une réduction considérable du risque, et elle découle d'un choix fonctionnel plutôt que technique.

Le droit d'accès. La cliente consulte à tout moment son profil, ses rendez-vous, ses commandes et l'intégralité de son historique de fidélité.

Le droit à l'effacement. C'est celui qui a demandé le plus de travail, et il fait l'objet de la section suivante.

4.8.2 L'effacement par anonymisation

Le droit à l'effacement se heurte ici à une exigence contradictoire : les commandes et les rendez-vous d'une cliente portent le chiffre d'affaires de l'institut, qui doit lui survivre. Une suppression en cascade détruirait la comptabilité ; un refus de supprimer méconnaîtrait la loi.

La solution retenue est l'**anonymisation** : la ligne survit, vidée de tout ce qui identifie la personne.

```
await this.prisma.$transaction([
  this.prisma.user.update({
    where: { id: userId },
    data: {
      // L'adresse d'origine est LIBEREE : si la cliente revient un jour, elle
      // pourra se reinscrire avec le meme email --sur un compte neuf. Le
      // domaine '.invalid' est reserve par la RFC 2606 et n'est routable nulle
      // part : aucun email ne partira jamais vers cette coquille.
      email: 'supprime-${userId}@compte-supprime.invalid',
      firstName: null, lastName: null, phone: null,
      passwordHash, // hache d'un secret aleatoire que nul ne connait
      emailVerifiedAt: null,
      deletedAt: new Date(),
    },
  }),
  // Les sessions ouvertes tombent tout de suite.
  this.prisma.refreshToken.deleteMany({ where: { userId } }),
  // Et un code de reinitialisation en cours ne doit pas servir de porte
  // derobee sur une coquille anonyme.
  this.prisma.verificationCode.deleteMany({ where: { userId } }),
]);
```

Listing 4.3 : Anonymisation d'un compte : ce qui est effacé, ce qui survit

Plusieurs points de ce traitement méritent d'être relevés.

- **L'opération est transactionnelle.** L'anonymisation, la destruction des sessions et celle des codes en cours réussissent ou échouent ensemble. Un échec partiel laisserait une coquille anonyme accessible par une session encore ouverte.
- **Le domaine `.invalid` est réservé par la RFC 2606** et n'est routable nulle part. Aucun courriel ne peut partir vers cette coquille, même par erreur de code — ce qu'un domaine inventé ne garantirait pas.
- **L'adresse d'origine est libérée**, ce qui permet une réinscription ultérieure sur un compte neuf.
- **Trois verrous indépendants** ferment le compte : le refus de connexion sur le marqueur de suppression, le mot de passe devenu inconnu de tous, et le rejet à chaque requête des jetons d'accès déjà émis.

- **La garantie est structurelle.** Les rendez-vous et les commandes sont rattachés à la cliente en mode **Restrict** : si du code tentait un jour une suppression, la contrainte le ferait échouer bruyamment au lieu de détruire l'historique en silence.

L'opération est enfin **idempotente** : rejouée sur un compte déjà anonymisé, elle répond succès sans rien réécrire — ce qui préserve la date d'origine, seule trace de la date à laquelle le droit a été exercé.

4.8.3 Gestion des secrets

Aucun secret ne figure dans le dépôt. Les fichiers d'environnement ne sont pas versionnés ; seuls leurs modèles le sont, documentés valeur par valeur.

La configuration est **validée au démarrage**, et l'API refuse de démarrer si elle est incohérente. Ce choix mérite d'être souligné : la solution alternative — démarrer et se débrouiller — produit des états dégradés que rien ne signale.

- **Le secret de signature doit compter au moins 32 caractères.** Un secret plus court se casse hors ligne, et permet ensuite de forger un jeton d'administration sans jamais toucher à l'API.
- **L'envoi réel de courriels est exigé en production.** Un serveur de production laissé en mode console n'enverrait aucun code de vérification ni de réinitialisation, et rien ne le signalerait : les clientes se retrouveraient simplement incapables de valider leur compte.
- **Les valeurs de repli sont bannies** là où elles seraient dangereuses. Le secret de signature est lu par une méthode qui lève une exception s'il manque, à la signature comme à la vérification.

4.8.4 Sauvegardes

Un script produit un cliché horodaté de la base et supprime ceux de plus de quatorze jours. Deux règles d'exploitation l'accompagnent, et elles comptent autant que le script : le cliché doit être recopié *hors de la machine* — une sauvegarde qui vit sur le serveur qu'elle sauvegarde ne protège de rien — et le dossier des images, qui vit hors de la base, doit être copié séparément. La procédure de restauration et sa vérification sont documentées dans le script lui-même.

4.9 Sécurité de la chaîne de développement

La sécurité d'une application ne s'arrête pas à son code : elle englobe ses dépendances, sa construction et son déploiement.

Les dépendances. Les versions sont figées par un fichier de verrouillage, et deux dépendances *transitives* sont forcées à des versions corrigées : une bibliothèque d'analyse YAML tirée par la couche de documentation, et une bibliothèque d'identifiants tirée par le générateur de classeurs Excel. Ces dépendances-là sont les plus faciles à négliger — on ne les a pas choisies, et elles n'apparaissent pas dans la liste des dépendances directes.

L'image de production. Elle est construite en trois étapes, si bien que l'image livrée ne contient ni compilateur, ni cadriciel de test, ni outillage de développement — une réduction de surface d'attaque autant qu'un gain de taille. Le processus y tourne sous un utilisateur non privilégié, et non en tant que `root`.

L'intégration continue. Chaque modification rejoue l'intégralité des tests contre une véritable base PostgreSQL. Plusieurs de ces tests portent directement sur des propriétés de sécurité : rotation et détection de réutilisation des jetons, anonymisation des comptes, capacité des créneaux sous concurrence, restriction des exports.

L'hygiène du dépôt. Les notes de travail internes, qui décrivaient des faiblesses avant leur correction, ont été retirées du suivi. Le retrait est documenté avec sa limite — il ne réécrit pas l'historique, et le contenu demeure accessible dans les commits passés. Cette honnêteté a une valeur opérationnelle : elle évite de tenir pour effacé ce qui ne l'est pas.

4.10 Synthèse

Le tableau ci-dessous récapitule, pour chaque menace du modèle, la contre-mesure retenue et l'endroit où elle est appliquée.

Tableau 4.6 : Synthèse menaces → contre-mesures

Menace	Contre-mesure
Force brute sur les mots de passe	Argon2id ; 10 tentatives par minute et par IP ; politique de complexité identique sur les trois chemins de saisie
Vol d'un jeton de session	Jeton d'accès de 15 minutes ; rafraîchissement haché, tourné, avec révocation de toute la famille en cas de réutilisation ; coffre sécurisé du téléphone
Devinette d'un code à 6 chiffres	Générateur cryptographique ; stockage haché ; 5 tentatives ; 15 minutes de validité ; comparaison en temps constant ; recherche par (compte, usage)
Énumération de comptes	Réponses identiques et <i>temps de réponse</i> identiques, par vérification factice sur adresse inconnue
Élévation de privilège	Garde de rôle côté serveur sur chaque route ; rôle relu en base à chaque requête ; rejet des propriétés non déclarées
Lecture des données d'autrui	Contrôle de propriété au niveau de la requête en base ; identifiants UUID
Jeton émis avant suppression	Compte rechargé et marqueur de suppression vérifié à chaque requête
Injection SQL	Requêtes paramétrées par l'ORM ; unique requête manuelle sans entrée utilisateur
XSS par téléversement	Reconnaissance par signature binaire ; SVG refusé ; nom aléatoire ; en-têtes nosniff et CSP
Fuite du schéma de base	Filtre d'exceptions global ; documentation OpenAPI coupée en production
Déni de service	Limitation de débit globale et par route ; pagination obligatoire ; plafond d'export à 20 000 lignes
Condition de course exploitable	Verrou consultatif sur les réservations ; écritures conditionnelles atomiques sur le stock et le solde
Interception du trafic	API sur interface locale ; HTTPS imposé par le reverse proxy ; HSTS ; garde-fou HTTPS côté mobile
Répudiation d'une opération	Motif obligatoire et auteur enregistré ; journal d'audit ; liens vers l'employée jamais supprimés en cascade
Atteinte aux données personnelles	Minimisation ; aucune donnée de paiement ; anonymisation transactionnelle ; sauvegardes horodatées

4.11 Limites connues et durcissement à venir

Aucun système n'est sûr dans l'absolu, et un rapport qui prétendrait le contraire manquerait son objet. Les limites suivantes sont assumées et identifiées.

L'absence de second facteur. L'authentification repose sur un seul facteur. Pour un compte cliente, l'enjeu reste mesuré ; pour le compte de direction, qui donne accès à l'ensemble du fichier clientes, un second facteur serait la première amélioration à apporter.

Les données au repos ne sont pas chiffrées. La base repose sur le chiffrement du disque du serveur hôte. Un chiffrement au niveau des colonnes les plus sensibles constituerait une couche supplémentaire.

Le compteur de débit vit en mémoire. Il est donc propre à chaque instance et remis à zéro au redémarrage. Tant que l'API tourne en un seul exemplaire, la protection est effective ; une mise à l'échelle horizontale imposerait de déporter ce compteur dans un magasin partagé.

Aucun audit externe n'a été mené. Les propriétés de sécurité sont établies par les tests automatisés et par la revue du code. Un test d'intrusion mené par un tiers reste la seule façon de découvrir ce que la personne qui a écrit le code ne pense pas à chercher.

La supervision est minimale. Les événements sont journalisés, mais rien ne les agrège ni ne déclenche d'alerte. Une série d'échecs de connexion sur un même compte devrait alerter, et ne le fait pas aujourd'hui.

Les points restant à faire avant l'ouverture publique. Le certificat HTTPS et le reverse proxy conditionnent plusieurs des mesures décrites ici : tant qu'ils ne sont pas en place, la protection du transport repose sur le fait que le service n'est pas encore exposé.

Conclusion

Ce chapitre a présenté la sécurité de la plate-forme comme une chaîne continue, du modèle de menaces jusqu'aux limites qui subsistent. Trois idées en résument la démarche.

D'abord, **la frontière de confiance passe à l'entrée de l'API**. Ni l'application mobile, ni le back-office, ni aucune valeur venue d'un client n'est cru sur parole ; ce qui

est masqué dans une interface n'est jamais protégé pour autant.

Ensuite, **les défauts sont fermés**. Une route est authentifiée et comptée sauf déclaration contraire, une propriété inconnue fait échouer la requête, et une configuration incohérente empêche le démarrage plutôt que de produire un état dégradé silencieux.

Enfin, **la sécurité n'a pas été ajoutée après coup**. Les mécanismes qui garantissent qu'un créneau n'est pas vendu deux fois sont les mêmes qui empêchent de dépenser deux fois les mêmes points ; le figement des montants sert à la fois l'exactitude comptable et la non-répudiation ; l'anonymisation concilie un droit légal et la survie de l'historique. Correction fonctionnelle et sécurité se sont révélées être, le plus souvent, deux vues du même problème.

Le chapitre suivant présente la mise en œuvre effective de la solution et les tests qui en établissent les propriétés.

CHAPITRE 5

Mise en œuvre

Introduction

Ce chapitre présente la réalisation effective de la solution. Il décrit d’abord la structure du code des trois applications, puis la stratégie de tests mise en place et son automatisation par intégration continue. Il présente ensuite les interfaces développées, illustrées par des captures d’écran, et se termine par les modalités de déploiement et d’exploitation de la plate-forme.

5.1 Structure du projet

Le projet est organisé en mono-dépôt : les trois applications vivent dans un même dépôt Git, à trois emplacements distincts. Ce choix évite la désynchronisation entre le contrat de l’API et ses consommateurs — une modification du serveur et son adaptation côté client sont validées ensemble par la même chaîne d’intégration continue.

Tableau 5.1 : Organisation du mono-dépôt

Dossier	Rôle	Pile technique
backend/	API REST, règles métier, base de données	NestJS · Prisma · PostgreSQL
backoffice/	Interface de gestion	React · Vite · Zustand
mobile/	Application cliente	Expo · React Native · i18next
scripts/	Sauvegarde de la base	Shell
brand/	Identité visuelle et génération des icônes	Python

5.1.1 Le backend

Le code de l'API représente environ 8 000 lignes, auxquelles s'ajoutent 5 150 lignes de tests. Il est découpé en quatorze modules NestJS, réunis par un module racine.

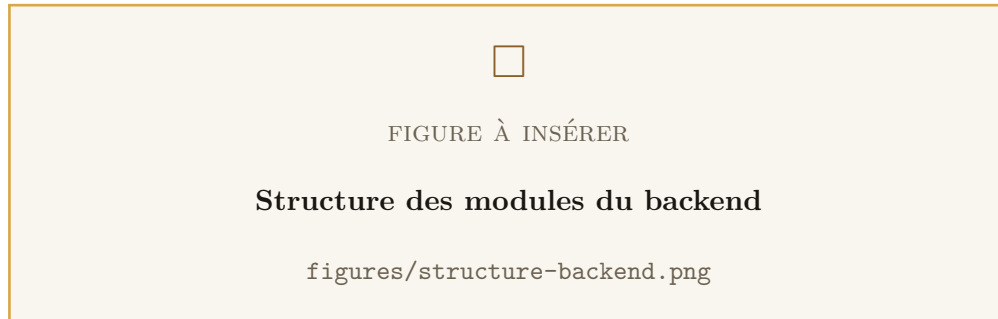


Figure 5.1 : Structure des modules du backend

Douze d'entre eux exposent un contrôleur et suivent tous la même organisation, ce qui rend le code prévisible :

- le **contrôleur** déclare les routes, leurs droits d'accès, leurs limites de débit et leur documentation ; il ne contient aucune règle ;
- le **service** porte la logique métier et les accès aux données ;
- les **objets de transfert** décrivent et valident les entrées ;
- les **tests** accompagnent le module qu'ils vérifient.

Les deux modules restants n'exposent aucune route : **prisma** met le client de base de données à la disposition des autres, et **mail** l'acheminement des courriels.

Trois dossiers, enfin, ne sont pas des modules mais du code partagé : **common** regroupe le filtre d'exceptions, l'intercepteur de traduction, la pagination et les validateurs communs, **config** valide les variables d'environnement au démarrage, et **i18n** porte les messages traduits de l'API.

Le cœur du système. Trois services concentrent l'essentiel de la complexité : le service des rendez-vous (693 lignes), qui porte le moteur de créneaux, la gestion du fuseau et le verrou de réservation ; le service de fidélité (837 lignes), qui porte les deux circuits de récompense et l'idempotence des crédits ; et le service d'authentification (477 lignes), qui porte la rotation des jetons et les défenses contre l'énumération de comptes.

5.1.2 Le back-office

Le back-office représente environ 6 500 lignes de TypeScript et React, organisées en huit pages et sept composants partagés.

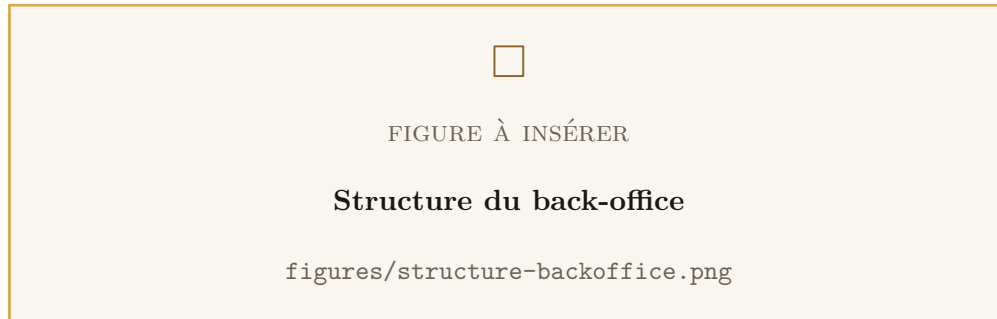


Figure 5.2 : Structure du back-office

- **pages/** — une page par domaine métier : tableau de bord, commandes, rendez-vous, catalogue, fidélité, horaires, utilisateurs, connexion ;
- **components/** — les éléments réutilisés : mise en page, pagination, boîte de confirmation, garde de route authentifiée, formulaire de catalogue, fenêtre d’export, fenêtre de réservation pour une cliente ;
- **api/** — une façade par domaine, seule à connaître les adresses des routes ; les pages ne manipulent jamais d’URL ;
- **stores/** — l’état global, réduit à la session authentifiée ;
- **theme/** — la charte graphique partagée.

Le client HTTP centralise deux comportements délicats : l’ajout automatique du jeton d’accès à chaque requête, et le rafraîchissement transparent de la session lorsqu’il expire — la requête ayant échoué est alors rejouée, sans que l’utilisatrice s’aperçoive de quoi que ce soit.

5.1.3 L’application mobile

L’application mobile représente environ 9 700 lignes, réparties en vingt-quatre écrans et dix-huit composants.

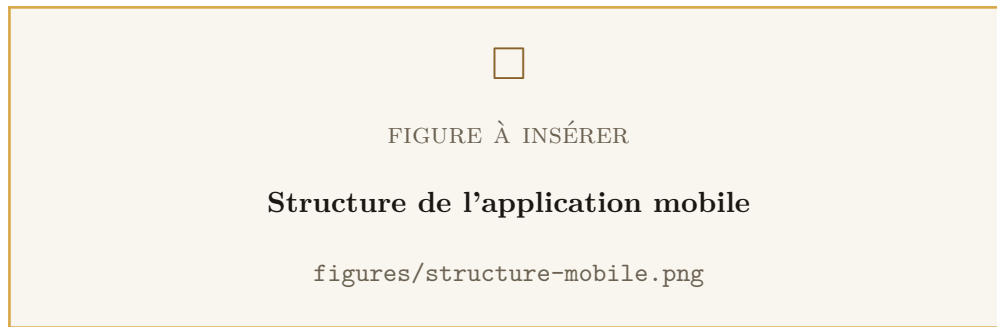


Figure 5.3 : Structure de l'application mobile

La navigation s'organise en une barre d'onglets — Accueil, Prestations, Produits, Fidélité, Profil — surmontant des piles de navigation propres à chaque section. Le panier et la session sont conservés dans des magasins d'état, le jeton de session étant stocké dans le coffre sécurisé du système d'exploitation plutôt qu'en clair.

L'application est traduite en trois langues, avec chargement de la langue du téléphone au démarrage. La prise en charge de l'arabe a demandé une attention particulière : au-delà de la traduction, elle impose de vérifier chaque écran dans le sens d'écriture de droite à gauche.

5.2 Tests et validation

5.2.1 Stratégie de tests

La stratégie retenue repose sur un constat simple : *les propriétés les plus importantes de ce projet ne sont pas démontrables par un test unitaire classique*. Qu'un calcul de points donne le bon résultat se vérifie avec une base simulée. Que trois réservations simultanées ne puissent pas occuper deux cabines ne se vérifie qu'avec une vraie base, un vrai verrou et de vraies transactions.

Les tests se répartissent donc en deux familles.

- **Les tests unitaires**, sur base simulée. Ils vérifient les règles métier pures : calcul des points, transitions d'états autorisées, génération de la grille de créneaux, validation des entrées, contrôle des droits.
- **Les tests d'intégration**, sur une véritable base PostgreSQL. Ils vérifient ce qu'une base simulée ne saurait démontrer : sérialisation des réservations concurrentes, atomicité du crédit de fidélité, annulation effective d'une transaction en échec, idempotence de l'émission des bons, rotation et détection de réutilisation des jetons, anonymisation des comptes.

Au total, le backend est couvert par 231 tests répartis en 25 suites, dont 8 écrivent réellement en base.

Tableau 5.2 : Répartition des suites de tests par domaine

Domaine	Suites	Base réelle	Propriétés vérifiées
Rendez-vous	5	1	Grille de créneaux, fuseau du centre, fermetures, machine à états, capacité sous concurrence
Fidélité	4	2	Calcul et idempotence des crédits, débit conditionnel du solde, paliers de visites, émission et remise des bons
Commandes	5	2	Contrôle et décrétement atomique du stock, restitution à l'annulation, crédit à la remise, machine à états
Authentification	3	1	Hachage, rotation des jetons, détection de réutilisation, validation du jeton d'accès
Utilisateurs	3	1	Profil, unicité de l'administratrice, anonymisation du compte supprimé
Exports	2	0	Génération des classeurs, restriction à la gérante, bornes de dates
Horaires	1	0	Refus des plages qui se chevauchent
Infrastructure	2	1	Connexion à la base, sonde de santé
Total	25	8	231 tests

5.2.2 Le test de concurrence, en détail

Le test le plus instructif du projet mérite d'être décrit, car il a directement dicté une décision d'architecture.

Il place le centre à une capacité de deux places, puis lance trois réservations *réellement simultanées* sur le même créneau. La propriété attendue est que deux réussissent et qu'une échoue — jamais trois.

```
// Trois réservations lancées en parallèle sur le même créneau,
// avec une capacité de 2 places.
const resultats = await Promise.allSettled([
  service.create(cliente1.id, { serviceId, startAt }),
  service.create(cliente2.id, { serviceId, startAt }),
```

```

    service.create(cliente3.id, { serviceId, startAt }),
  });

  const reussies = resultats.filter((r) => r.status === 'fulfilled');
  expect(reussies).toHaveLength(2); // et jamais 3

```

Listing 5.1 : Principe du test de concurrence sur les réservations

Sur l’implémentation initiale — compter les rendez-vous du créneau, puis insérer — ce test échouait de façon reproductible : les trois réservations étaient acceptées. Les trois comptaient avant qu’aucune n’ait inséré, et les trois trouvaient de la place. C’est ce test, et non une revue de code, qui a imposé le recours au verrou consultatif décrit au chapitre 2.

Ces tests partagent la même base et le même verrou : ils doivent donc être exécutés en série, ce qui est imposé par une option dédiée du lanceur de tests.

5.2.3 Intégration continue

L’ensemble est rejoué automatiquement à chaque modification par une chaîne GitHub Actions. Elle comporte trois travaux.

Tableau 5.3 : Travaux de la chaîne d’intégration continue

Travail	Étapes
Backend	Installation des dépendances, génération du client Prisma, application des migrations sur une base PostgreSQL éphémère, exécution du script d’amorçage, compilation, puis exécution des 25 suites de tests.
Back-office	Installation des dépendances et vérification de types.
Mobile	Installation des dépendances et vérification de types.

Un point de cette chaîne mérite d’être signalé, car il n’est pas évident : le script d’amorçage y est indispensable. Il crée les horaires d’ouverture, et sans une seule ligne dans cette table, le centre est considéré fermé tous les jours — toute la suite de tests des rendez-vous échouerait pour une raison qui n’a rien à voir avec le code testé.

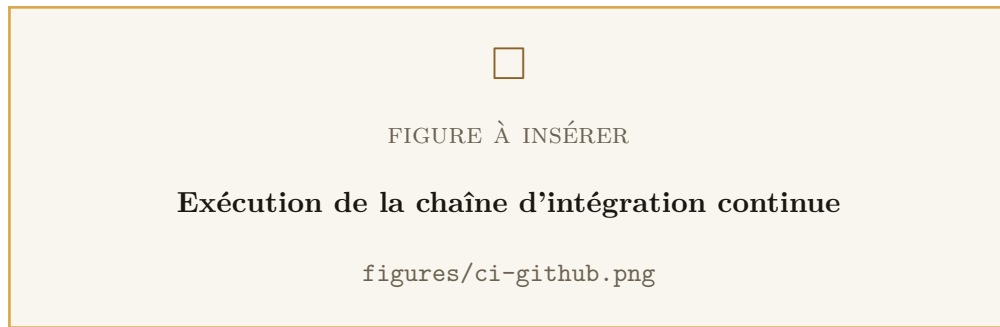


Figure 5.4 : Exécution de la chaîne d'intégration continue

5.2.4 Résultats



Figure 5.5 : Résultat de l'exécution complète de la suite de tests

L'exécution complète est verte : 25 suites, 231 tests, aucun échec. Le back-office et l'application mobile, qui n'ont pas de tests automatisés, sont couverts par la vérification de types — un filet plus mince, mais qui détecte effectivement toute divergence entre le contrat de l'API et ses consommateurs.

5.3 Interfaces graphiques

5.3.1 Application mobile

5.3.1.1 Authentification



Figure 5.6 : Écran de connexion · Écran d'inscription

L'inscription recueille l'adresse, le mot de passe, le nom, le prénom et le téléphone. Ces trois derniers champs sont obligatoires : un rendez-vous ou une commande sans nom ni numéro de téléphone est inexploitable pour l'institut, qui doit pouvoir rappeler la cliente. Un code de vérification est envoyé par e-mail à l'issue de l'inscription.

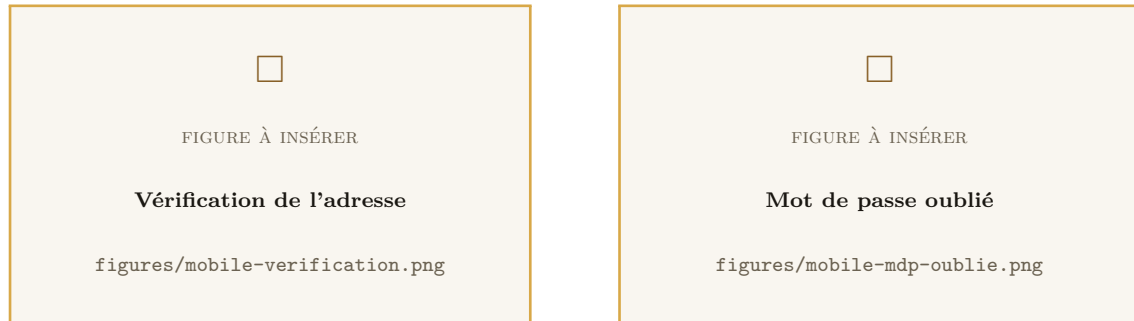


Figure 5.7 : Vérification de l'adresse · Mot de passe oublié

5.3.1.2 Accueil et catalogues

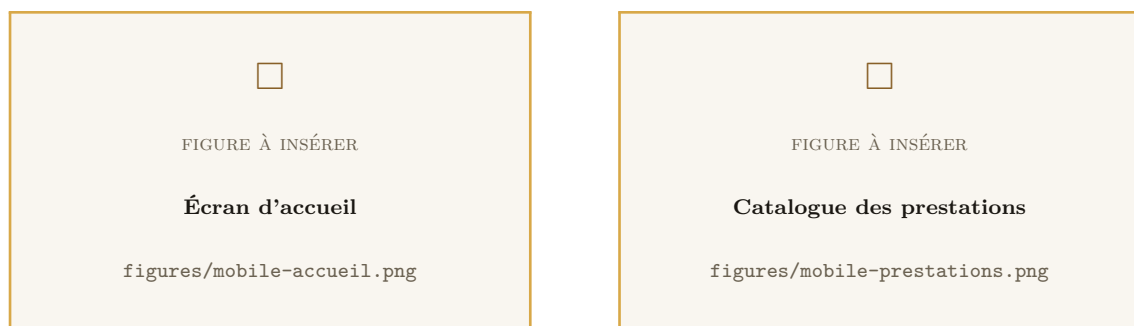


Figure 5.8 : Écran d'accueil · Catalogue des prestations

L'accueil réunit en une vue le solde de fidélité, une sélection de prestations et une sélection de produits. C'est l'écran le plus consulté, et celui dont le coût en requêtes a été le plus surveillé : il ne demande que ce qu'il affiche.



Figure 5.9 : Détail d'une prestation · Catalogue des produits

5.3.1.3 Réservation



Figure 5.10 : Choix de la date et du créneau · Récapitulatif avant validation

L'écran de réservation affiche la grille des créneaux du jour choisi. Les créneaux indisponibles — capacité atteinte, ou heure déjà passée — restent visibles mais désactivés, plutôt que masqués : une cliente qui ne voit pas le créneau de 15 h ignore s'il est pris ou si l'institut n'ouvre pas à cette heure-là.

Un jour fermé, qu'il le soit par les horaires hebdomadaires ou par une fermeture exceptionnelle, est annoncé explicitement.



Figure 5.11 : Confirmation du rendez-vous · Mes rendez-vous

5.3.1.4 Commande de produits

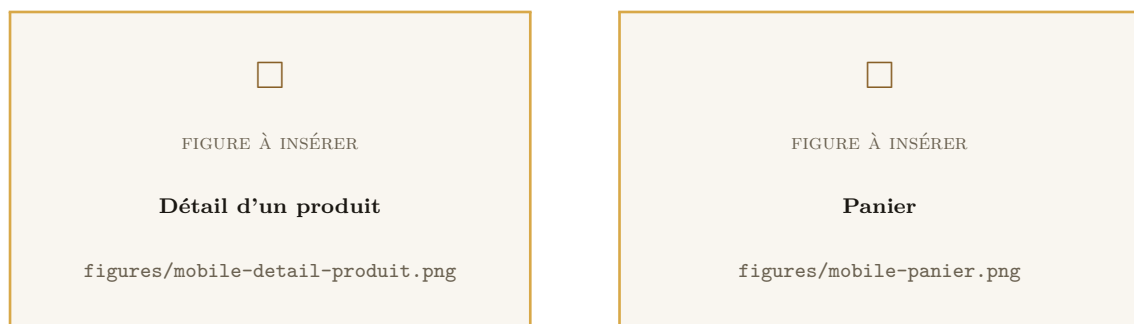


Figure 5.12 : Détail d'un produit · Panier



Figure 5.13 : Validation de commande · Mes commandes

La validation propose le retrait à l’institut ou la livraison à domicile, laquelle exige une adresse. Le paiement s’effectue à la remise. La cliente peut annuler sa commande tant qu’elle ne lui a pas été remise.

5.3.1.5 Fidélité



Figure 5.14 : Écran de fidélité · Mes récompenses et bons

L’écran de fidélité réunit le solde de points, la progression vers le prochain palier de visites, le catalogue des récompenses échangeables et l’historique des mouvements. L’écran des récompenses présente les bons dus : ceux offerts par un palier et ceux obtenus par échange de points, avec leur code à présenter au comptoir.

5.3.1.6 Profil

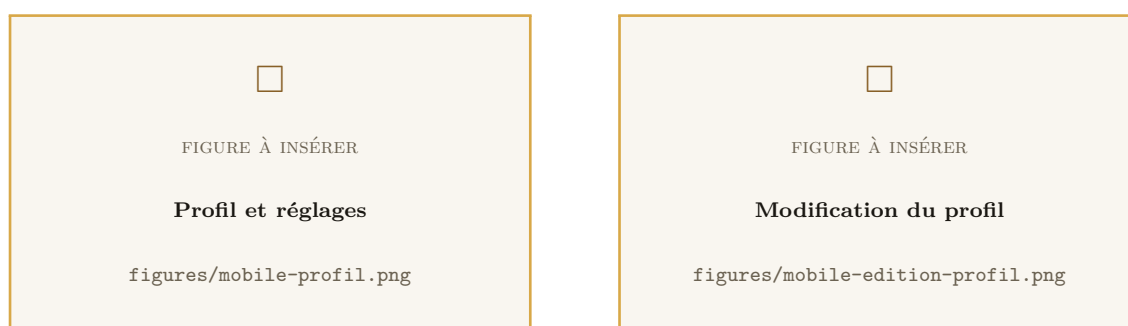


Figure 5.15 : Profil et réglages · Modification du profil

Les réglages permettent de changer de langue, de basculer entre les modes clair et sombre, de modifier le mot de passe et de supprimer le compte.

5.3.2 Back-office

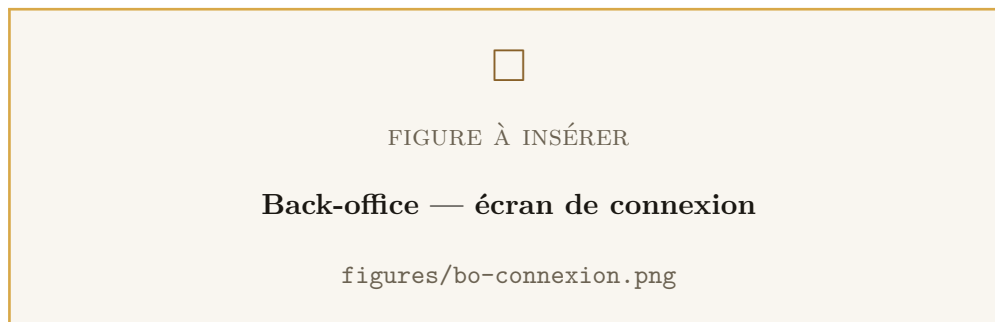


Figure 5.16 : Back-office — écran de connexion

Une cliente est refusée à l'entrée du back-office : ce contrôle est effectué par le serveur et non par l'interface.

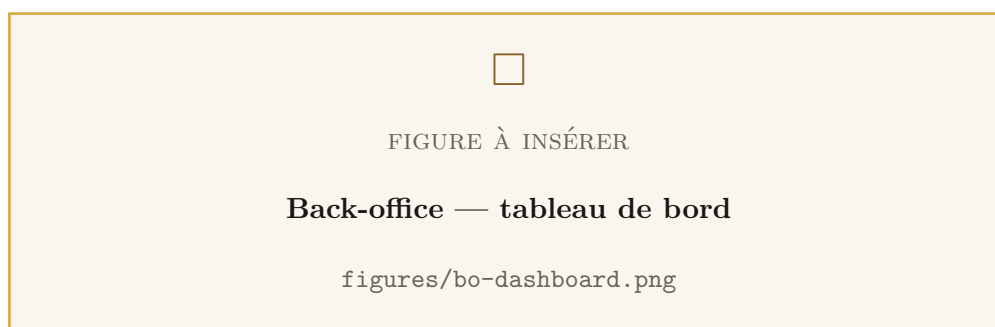


Figure 5.17 : Back-office — tableau de bord

Le tableau de bord réunit ce qui appelle une action immédiate : les commandes en attente de confirmation, les rendez-vous du jour et les produits en stock faible. La notion d'« aujourd'hui » y est calculée par le serveur, dans le fuseau du centre : c'est lui qui fait autorité, et non le poste qui consulte.

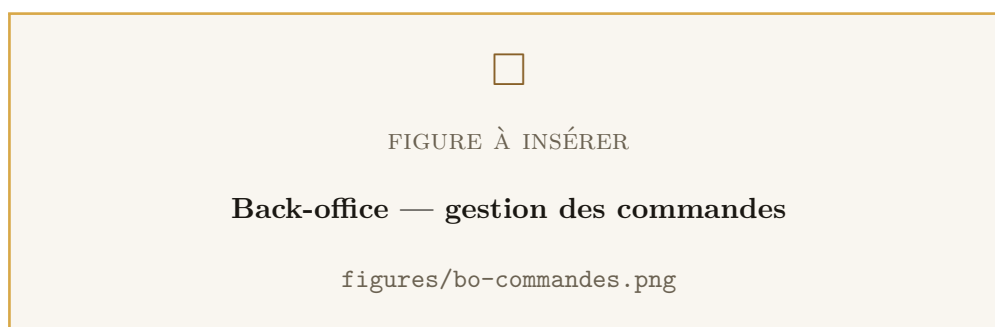


Figure 5.18 : Back-office — gestion des commandes

Les commandes sont présentées par statut. Chaque changement de statut est soumis à la machine à états : les seules transitions proposées sont celles que le serveur accepterait.

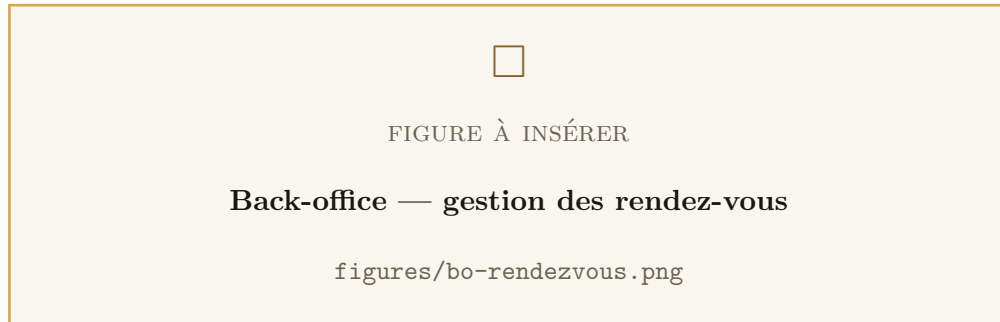


Figure 5.19 : Back-office — gestion des rendez-vous

Cette page permet également de réserver pour le compte d'une cliente — c'est-à-dire de saisir un rendez-vous pris par téléphone — et de reporter un rendez-vous existant, avec les mêmes contrôles de disponibilité que ceux appliqués à une réservation cliente.

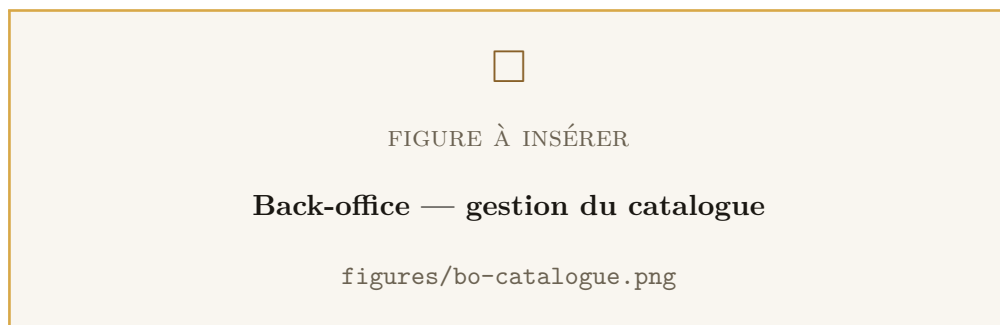


Figure 5.20 : Back-office — gestion du catalogue

Le catalogue couvre les prestations, les produits et leurs catégories, y compris le téléversement des photos et le suivi des stocks. Un élément retiré du catalogue est désactivé et non supprimé : il disparaît immédiatement des applications clientes, mais reste rattaché à l'historique des commandes et des rendez-vous qui le référencent.

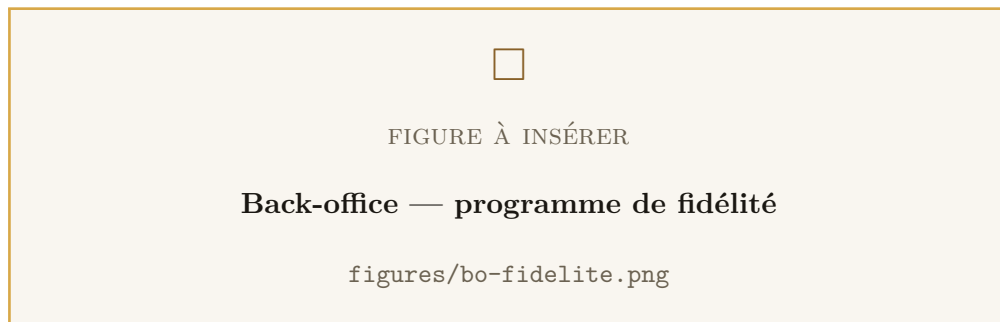


Figure 5.21 : Back-office — programme de fidélité

Cette page réunit le catalogue des récompenses, les paliers de visites, la liste des bons dus au comptoir, l'ajout manuel de points — motif obligatoire — et le journal d'audit de ces ajouts, consultable par la seule gérante.

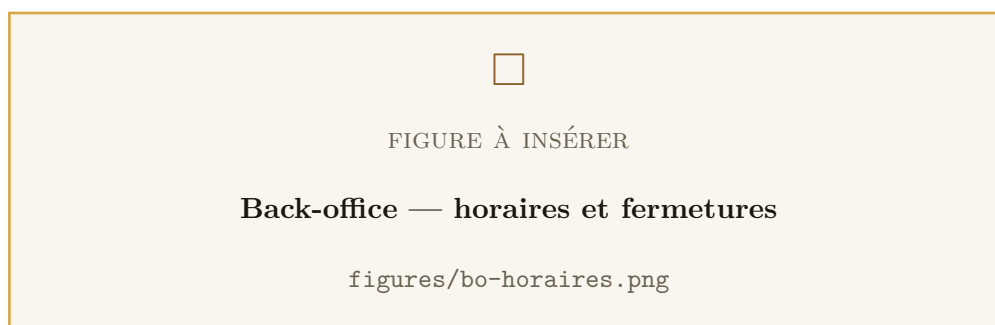


Figure 5.22 : Back-office — horaires et fermetures

Les horaires se règlent par plages, à raison de plusieurs plages par jour. Un jour sans aucune plage est fermé. Les fermetures exceptionnelles — congés, jours fériés — priment sur les horaires hebdomadaires. La capacité du centre et l'écart entre créneaux se règlent depuis cette même page, sans redéploiement.

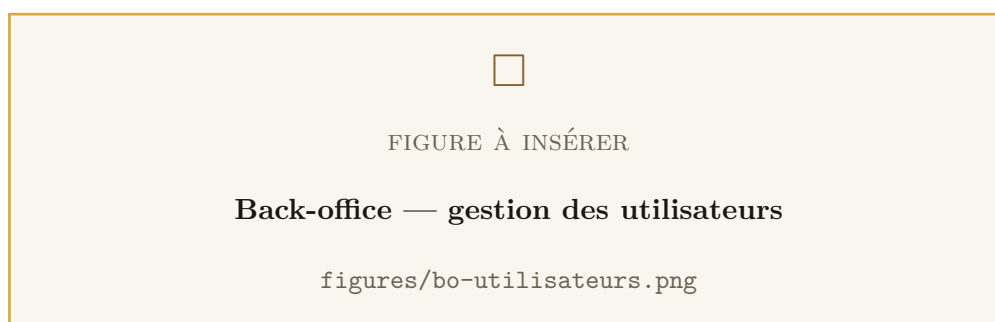


Figure 5.23 : Back-office — gestion des utilisateurs

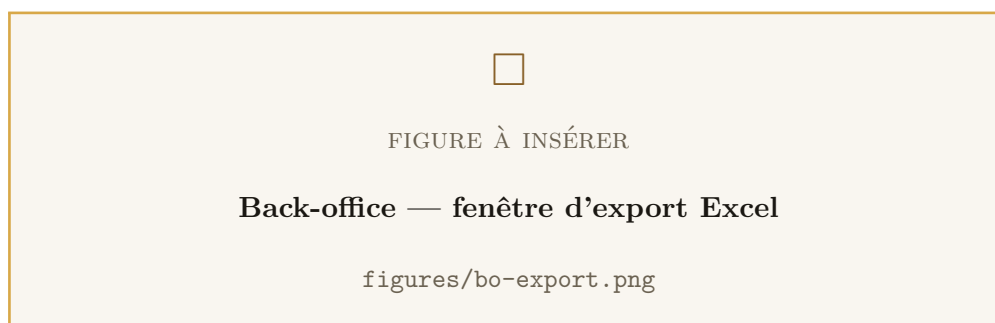


Figure 5.24 : Back-office — fenêtre d'export Excel

Les exports produisent des classeurs Excel des clientes, des commandes et des rendez-vous, sur une période choisie. Le fichier est destiné à la gérante et non à un développeur :

aucune valeur technique brute n’y apparaîât — les statuts y sont écrits en toutes lettres — et les montants y restent des *nombres* plutôt que du texte, de sorte qu’une somme ou un tableau croisé fonctionne dans le classeur.

5.3.3 Courriers électroniques



Figure 5.25 : Courriel de vérification d’adresse

Les courriels sont envoyés dans la langue choisie par la cliente. En développement, aucun message ne part : les codes s’affichent dans les journaux du serveur, ce qui permet de dérouler l’intégralité des parcours — inscription, vérification, mot de passe oublié — sans dépendre d’un fournisseur de messagerie.

5.4 Déploiement et exploitation

5.4.1 Mise en production

La pile complète se lance par une commande unique, qui enchaîne la base de données, l’application des migrations et le démarrage de l’API. La documentation interactive de l’API est automatiquement coupée en production.

5.4.2 Sauvegardes

Un script produit un cliché horodaté de la base et supprime ceux de plus de quatorze jours, à automatiser par une tâche planifiée. Les deux règles d’exploitation qui l’accompagnent — recopier le cliché hors de la machine, et copier séparément le dossier des images, qui vit hors de la base — ont été exposées au chapitre 4 : elles relèvent de la disponibilité des données autant que de l’exploitation courante.

La procédure de restauration, et la vérification qui doit la suivre, sont documentées dans le script lui-même — à l’endroit où on la cherchera le jour où elle servira.

5.4.3 Ce qui reste à faire avant une ouverture publique

Par honnêteté sur l’état de la livraison, trois points restent ouverts :

- l'acquisition d'un nom de domaine et d'un certificat HTTPS ;
- la mise en place d'un serveur de messagerie définitif, à la place de la solution provisoire actuelle ;
- les notifications *push*, qui dépendent d'une compilation native, donc du nom de domaine.

Conclusion

Ce chapitre a présenté la mise en œuvre de la solution : la structure du code des trois applications, la stratégie de tests et son automatisation, les interfaces réalisées et les modalités d'exploitation. La démonstration du produit final à travers ses écrans montre une plate-forme complète et cohérente ; les 231 tests qui l'accompagnent, dont ceux qui écrivent dans une véritable base, en établissent les propriétés les plus difficiles.

Conclusion Générale et Perspectives

Bilan du travail réalisé

Ce projet de fin d'année a abouti à une plate-forme logicielle complète pour *Yani Concept by Fati* : une API REST portant l'intégralité des règles métier, une application mobile trilingue destinée aux clientes, et un back-office web destiné à l'institut. Les trois applications partagent une même base PostgreSQL et un même langage, et sont validées ensemble par une chaîne d'intégration continue.

Les objectifs fixés au départ ont été atteints. La réservation est possible à toute heure, sur une grille de créneaux calculée à partir des horaires réels du centre. Le carnet papier est remplacé par une base sauvegardée, consultable depuis plusieurs postes. La carte de fidélité tamponnée a laissé place à un compte doté d'un solde, d'un historique, de paliers de visites et de bons traçables. Le stock est suivi, contrôlé et décrémenté au bon moment. La gérante dispose enfin d'une visibilité sur son activité et de l'export de ses données.

Le travail ne s'est cependant pas limité à la mise en œuvre de fonctionnalités. Sa part la plus exigeante a porté sur des propriétés qui ne se voient pas à l'écran :

- **la correction sous concurrence** — un créneau n'est jamais vendu deux fois, une unité de stock jamais attribuée deux fois, un point de fidélité jamais crédité deux fois ; ces propriétés sont établies par des tests qui écrivent dans une véritable base PostgreSQL, parce qu'aucune base simulée ne saurait les démontrer ;
- **la fidélité au temps réel** — les instants sont stockés en temps universel, les horaires exprimés en heure locale du centre, et la conversion n'est faite qu'à un seul endroit, par le serveur, qui possède la base des fuseaux ;
- **la conservation de l'historique** — une cliente peut exercer son droit à l'effacement sans que le chiffre d'affaires qu'elle a généré ne disparaisse avec elle ; les modes de suppression déclarés dans le schéma rendent cette garantie structurelle plutôt que déclarative ;

- **la sécurité** — traitée à partir d'un modèle de menaces explicite et non comme une couche ajoutée en fin de parcours : hachage Argon2id, jetons courts, rotation avec révocation de famille en cas de réutilisation détectée, uniformisation du temps de réponse contre l'énumération de comptes, autorisation vérifiée par le serveur sur chaque route et doublée d'un contrôle de propriété des ressources, validation stricte des entrées, reconnaissance des fichiers téléversés par leur signature binaire.

Apports personnels

Sur le plan technique, ce projet a été l'occasion de mener seul, de bout en bout, une réalisation à trois cibles : un serveur, une application web et une application mobile. Il m'a permis d'approfondir des sujets que le cursus aborde nécessairement de façon théorique — la concurrence en base de données, la gestion des fuseaux horaires, la sécurité applicative — et de découvrir qu'ils ne se comprennent réellement qu'au moment où un test les met en échec.

L'apport le plus net concerne la sécurité, et il tient moins à des techniques qu'à un changement de regard. Appliquer une grille de menaces à un système que l'on a soi-même écrit oblige à l'examiner du point de vue de quelqu'un qui cherche à en abuser, et non de celui qui l'utilise comme prévu. C'est cet exercice qui a fait apparaître ce qu'aucune relecture fonctionnelle ne signalait : que masquer un bouton n'a jamais protégé la route correspondante, qu'un temps de réponse suffit à trahir l'existence d'un compte, et qu'une condition de course déclenchable à volonté n'est pas un défaut de correction mais une vulnérabilité.

Le test de concurrence sur les réservations en est l'illustration la plus nette. L'implémentation initiale — compter puis insérer — paraissait évidemment correcte à la lecture. Il a fallu trois réservations réellement simultanées pour montrer qu'elle ne l'était pas. C'est l'enseignement que je retiens le plus fermement de ce projet : *en matière de concurrence, l'intuition ne vaut rien, et seul un test qui reproduit la simultanéité fait autorité.*

Sur le plan méthodologique, la démarche agile a démontré son intérêt de façon concrète. Plusieurs des règles les plus importantes du système — qu'un rendez-vous occupe un créneau fixe et non la durée de sa prestation, qu'une réclamation de récompense doive laisser une trace au comptoir, qu'un produit retiré du catalogue ne doive pas s'effacer de l'historique — ne figuraient dans aucune spécification initiale. Elles sont apparues en montrant des versions fonctionnelles à la personne qui allait s'en servir.

Sur le plan personnel, travailler pour une utilisatrice unique, identifiée et non infor-

maticienne a été formateur. Un message d'erreur incompréhensible n'est pas un détail lorsque la personne qui le reçoit n'a personne à qui le transmettre. Cette contrainte a orienté un grand nombre de décisions, du libellé des statuts dans les exports Excel jusqu'au choix d'afficher les créneaux indisponibles plutôt que de les masquer.

Difficultés rencontrées

Trois difficultés méritent d'être mentionnées, ainsi que la façon dont elles ont été surmontées.

La concurrence sur les réservations. Décrite plus haut, elle a imposé de descendre au niveau du SGBD et de recourir à un verrou consultatif PostgreSQL — une primitive qui ne figure dans aucune abstraction d'ORM.

Les fuseaux horaires. Les erreurs de fuseau sont silencieuses : elles ne lèvent aucune exception et produisent des résultats plausibles mais faux, décalés d'une heure. La convention a donc été fixée explicitement — stockage en temps universel, horaires en heure locale, conversion en un seul endroit — et documentée dans le code comme dans l'API.

Les pièges de déploiement invisibles. Plusieurs réglages d'exploitation ne cassent rien immédiatement, mais produisent des symptômes déroutants une fois en service : le réglage du proxy de confiance, sans lequel tout l'institut partage un seul compteur de requêtes ; la liste des origines autorisées, dont l'oubli n'apparaît que dans la console du navigateur. Ces pièges ont été documentés à l'endroit où ils se manifestent, avec leur symptôme et leur cause, plutôt que dans une documentation séparée que personne ne consulte au moment utile.

Perspectives

La plate-forme livrée est fonctionnelle, mais plusieurs évolutions ont été identifiées, classées ici par horizon.

À court terme — finaliser la mise en service.

- Acquisition d'un nom de domaine et d'un certificat HTTPS, avec le reverse proxy correspondant.
- Mise en place d'un serveur de messagerie définitif.
- Publication de l'application sur les magasins Android et iOS.

À moyen terme — enrichir l’usage.

- **Notifications *push*** : rappel de rendez-vous la veille, avis de commande prête, alerte de palier de fidélité franchi. C’est l’évolution la plus demandée, et elle dépend d’une compilation native, donc du nom de domaine.
- **Paiement en ligne**, en complément du paiement à la remise.
- **Statistiques de pilotage** : chiffre d’affaires par période, prestations les plus demandées, taux de retour de la clientèle. Les données sont déjà en base ; il ne manque que leur restitution.
- **Gestion fine des ressources** : aujourd’hui, la capacité du centre est un nombre unique de places réservables. Distinguer les cabines et les esthéticiennes permettrait de proposer des créneaux différenciés selon la prestation.

À moyen terme — durcir. Les limites de sécurité inventoriées au chapitre 4 appellent quatre chantiers, par ordre de priorité décroissante : un **second facteur d’authentification** sur le compte de direction, qui donne accès à l’ensemble du fichier clientes ; un **test d’intrusion externe**, seule façon de découvrir ce que la personne qui a écrit le code ne pense pas à chercher ; une **supervision** qui alerte sur les séries d’échecs de connexion plutôt que de se contenter de les journaliser ; et le report du compteur de débit dans un magasin partagé, préalable à toute mise à l’échelle horizontale.

À plus long terme.

- **Fiche cliente** tenue par l’institut : historique des soins, allergies signalées, préférences — un dossier de suivi qui prolongerait la relation personnelle qui fait la valeur du centre.
- **Gestion multi-établissements**, si l’institut venait à ouvrir un second point de vente.
- **Automatisation du marketing** : relance des clientes non revenues depuis un certain temps, offres d’anniversaire.

Au terme de ce travail, la plate-forme est prête à entrer en service. Elle ne remplace pas la relation entre l’institut et ses clientes — elle ne le cherche pas — mais elle décharge la gérante de ce qui l’en éloignait : le téléphone qui sonne pendant un soin, le carnet à consulter, la carte de fidélité à retrouver. C’est, en définitive, la mesure la plus juste de sa réussite.

Webographie

Les ressources ci-dessous ont été consultées durant la réalisation du projet. Les dates indiquées correspondent à la dernière consultation.

1. **NestJS** — documentation officielle du cadriciel de l'API (modules, gardes, intercepteurs, filtres d'exceptions, limitation de débit).
<https://docs.nestjs.com/>
2. **Prisma** — documentation de l'ORM : schéma, migrations, transactions, modes de suppression référentielle.
<https://www.prisma.io/docs>
3. **PostgreSQL** — documentation des fonctions de verrouillage consultatif, primitive au cœur du moteur de réservation.
<https://www.postgresql.org/docs/current/explicit-locking.html>
4. **React** — documentation officielle de la bibliothèque d'interface du back-office.
<https://react.dev/>
5. **React Native** — documentation officielle du cadriciel de l'application mobile.
<https://reactnative.dev/docs/getting-started>
6. **Expo** — documentation de la chaîne d'outils mobile : stockage sécurisé, polices, images, compilation native.
<https://docs.expo.dev/>
7. **TypeScript** — manuel de référence du langage.
<https://www.typescriptlang.org/docs/>
8. **Vite** — documentation de l'outil de construction du back-office.
<https://vite.dev/guide/>

9. **OWASP Top 10** — classement des dix risques de sécurité applicative les plus critiques ; grille de référence du chapitre 4.
<https://owasp.org/www-project-top-ten/>
10. **OWASP Cheat Sheet Series** — *Password Storage, Authentication, Session Management, File Upload* et *Mass Assignment* : recommandations suivies pour le hachage des mots de passe, la défense contre l'énumération de comptes, la rotation des jetons et le contrôle des fichiers téléversés.
<https://cheatsheetseries.owasp.org/>
11. **OWASP ASVS** — *Application Security Verification Standard* : référentiel de vérification employé pour établir la synthèse des contre-mesures.
<https://owasp.org/www-project-application-security-verification-standard/>
12. **Microsoft — Threat Modeling (STRIDE)** — description du modèle de menaces appliqué au chapitre 4.
<https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats>
13. **MITRE CWE** — *Common Weakness Enumeration* : nomenclature des faiblesses logicielles citées (IDOR, injection, condition de course, exposition d'informations).
<https://cwe.mitre.org/>
14. **JSON Web Tokens** — introduction au format et à ses usages.
<https://jwt.io/introduction>
15. **RFC 6819** — *OAuth 2.0 Threat Model and Security Considerations* : source de la stratégie de rotation des jetons de rafraîchissement et de la détection de réutilisation.
<https://datatracker.ietf.org/doc/html/rfc6819>
16. **Argon2** — spécification de la fonction de hachage retenue (RFC 9106).
<https://datatracker.ietf.org/doc/html/rfc9106>
17. **RFC 2606** — noms de domaine réservés, dont le domaine `.invalid` employé lors de l'anonymisation des comptes.
<https://datatracker.ietf.org/doc/html/rfc2606>

18. **Helmet** — documentation des en-têtes de sécurité HTTP posés par l'API.
<https://helmetjs.github.io/>
19. **MDN — HTTP security headers** — référence sur HSTS, CSP, X-Content-Type-Options, CORS et CORP.
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers>
20. **IANA Time Zone Database** — base des fuseaux horaires, référence de la zone Africa/Casablanca.
<https://www.iana.org/time-zones>
21. **date-fns** — documentation de la bibliothèque de manipulation de dates et de son extension pour les fuseaux horaires.
<https://date-fns.org/docs/Getting-Started>
22. **OpenAPI / Swagger** — spécification du format de documentation d'API et de son interface interactive.
<https://swagger.io/specification/>
23. **Docker** — documentation de la conteneurisation et de Docker Compose.
<https://docs.docker.com/>
24. **GitHub Actions** — documentation de la chaîne d'intégration continue.
<https://docs.github.com/en/actions>
25. **Jest** — documentation du cadriciel de tests.
<https://jestjs.io/docs/getting-started>
26. **Scrum Guide** — guide de référence de la méthode SCRUM.
<https://scrumguides.org/>
27. **UML** — spécification du langage de modélisation unifié publiée par l'OMG.
<https://www.omg.org/spec/UML/>
28. **Lucidchart** — plate-forme de création des diagrammes UML du présent rapport.
<https://www.lucidchart.com/>
29. **CNDP** — Commission Nationale de contrôle de la protection des Données à caractère Personnel : loi 09-08, obligations du responsable de traitement et droits des personnes concernées (accès, rectification, opposition, effacement).
<https://www.cndp.ma/>

30. **i18next** — documentation de la bibliothèque d'internationalisation employée côté mobile.

<https://www.i18next.com/>